

TRƯỜNG CƠ KHÍ – ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA CƠ ĐIỆN TỬ



ĐỒ ÁN TỐT NGHIỆP

**NGHIÊN CỨU VÀ MÔ PHỎNG HỆ THỐNG KIỂM TRA
CHẤT LƯỢNG PCB TỰ ĐỘNG TRONG DÂY CHUYỀN
SẢN XUẤT ĐIỆN TỬ SỬ DỤNG COMPUTER VISION
VÀ AI**

Giảng viên hướng dẫn: GS.TS. VŨ TOÀN THẮNG

Sinh viên thực hiện: NGUYỄN MINH HOÀNG

MSSV : 20240929E

Lớp : KSCS-K69

Hà Nội, tháng ... năm 2026

LỜI NÓI ĐẦU

Trong sản xuất điện tử, kiểm tra chất lượng PCB là công đoạn ảnh hưởng trực tiếp đến độ ổn định của sản phẩm sau lắp ráp. Với các bo mạch có nhiều linh kiện nhỏ, việc kiểm tra bằng mắt thường dễ bỏ sót lỗi, nhất là khi sản lượng tăng hoặc thao tác lắp lại trong thời gian dài. Vì vậy, việc xây dựng một quy trình kiểm tra tự động bằng camera và mô hình nhận dạng ảnh là hướng tiếp cận phù hợp với bài toán kiểm tra PCB trong dây chuyền hiện đại.

Đề tài này tập trung vào việc mô phỏng một hệ thống kiểm tra và phân loại PCB ESP32 DevKit trên Unity 3D. Hệ thống gồm các khối chính như băng tải mô phỏng, điểm kiểm tra, camera, mô hình YOLO Detection và khối phân loại sản phẩm. Trong đồ án, Unity được dùng để biểu diễn và kiểm chứng logic vận hành của hệ thống; chuyển động của PCB và quá trình phân loại được điều khiển bằng script C#, không đi sâu mô phỏng vật lý chi tiết của băng tải hay cơ cấu xi lanh.

Về phân dữ liệu và mô hình nhận dạng, đề tài khảo sát hai hướng: YOLO Classification kết hợp WGAN-GP và YOLO Detection kết hợp WGAN-GP. Sau khi phân tích đặc điểm đầu ra của từng hướng, YOLO Detection được chọn làm mô hình chính vì có thể trả về loại lỗi, độ tin cậy và vị trí linh kiện bị thiếu trên ảnh. WGAN-GP được giữ ở vai trò hỗ trợ mở rộng dữ liệu trong điều kiện số lượng ảnh lỗi còn hạn chế, không phải là thành phần quyết định chính của hệ thống.

Nội dung đồ án được trình bày trong 5 chương:

- Chương 1: Trình bày bối cảnh, lý do chọn đề tài, mục tiêu, phạm vi và định hướng nghiên cứu.
- Chương 2: Trình bày các cơ sở lý thuyết liên quan đến PCB ESP32, Computer Vision, WGAN-GP, YOLO và môi trường Unity 3D.
- Chương 3: Mô tả quá trình xây dựng dữ liệu, huấn luyện mô hình và thiết kế logic mô phỏng hệ thống kiểm tra - phân loại PCB.
- Chương 4: Trình bày kết quả thực nghiệm, bao gồm kết quả nhận dạng của mô hình và kết quả vận hành của hệ thống mô phỏng.
- Chương 5: Tổng kết, nêu hạn chế và đề xuất hướng phát triển tiếp theo.

BẢNG DUYỆT ĐỒ ÁN VÀ XÁC NHẬN
CHO SINH VIÊN NỘP ĐỒ ÁN TỐT NGHIỆP

Họ và tên sinh viên: Nguyễn Minh HoàngMã SV: 20240929E

Tên đề tài: Nghiên cứu và mô phỏng hệ thống kiểm tra chất lượng PCB tự động trong dây chuyền sản xuất điện tử sử dụng Computer Vision và AI

STT	Hạng mục	Nội dung	Đánh giá	
			ĐẠT	CHƯA ĐẠT
1	Bố cục đồ án	Bố trí trang bìa, trang lót, đúng mẫu quy định		
		Có lời mở đầu, lời cảm ơn		
		Mục lục chính xác, đúng mẫu		
		Danh mục bảng biểu, hình vẽ đầy đủ, chính xác		
		Danh mục cụm từ viết tắt đầy đủ, chính xác		
		Bố cục các chương đúng mẫu, có kết luận của mỗi chương		
		Phụ lục trình bày hợp lý (nếu có)		
		Danh mục tài liệu tham khảo đủ		
2	Nội dung đồ án	Nội dung của đồ án đã được GVHD duyệt, và kết luận: ☐ Đồng ý cho SV nộp đồ án cho Bộ môn để bảo vệ ☐ Không đồng ý cho SV nộp đồ án.		
3	Điểm hướng dẫn đồ án tốt nghiệp:			

Hà Nội, ngày ... tháng ... năm 2026

Giáo viên hướng dẫn

(Ký và ghi rõ họ tên)

LỜI CẢM ƠN

Trong quá trình học tập và thực hiện đồ án, em đã nhận được sự giúp đỡ của thầy cô, gia đình và bạn bè. Em xin gửi lời cảm ơn chân thành đến những người đã hỗ trợ em trong suốt thời gian thực hiện đề tài.

Em xin cảm ơn các thầy cô trong Khoa Cơ điện tử đã truyền đạt cho em những kiến thức nền tảng về cơ khí, điện tử, điều khiển, tự động hóa và lập trình. Những kiến thức này là cơ sở để em có thể xây dựng và hoàn thiện hệ thống mô phỏng trong đồ án.

Đặc biệt, em xin cảm ơn GS.TS. Vũ Toàn Thắng đã trực tiếp hướng dẫn, góp ý và định hướng cho em trong quá trình thực hiện đề tài. Những nhận xét của thầy giúp em nhìn rõ hơn phạm vi nghiên cứu, cách trình bày báo cáo và các vấn đề cần làm rõ khi xây dựng hệ thống kiểm tra PCB tự động.

Em cũng xin cảm ơn gia đình và bạn bè đã động viên, hỗ trợ và tạo điều kiện để em tập trung hoàn thành đồ án.

Do kiến thức và kinh nghiệm thực tế của bản thân còn hạn chế, đồ án chắc chắn vẫn còn những điểm cần tiếp tục hoàn thiện. Em mong nhận được góp ý của quý thầy cô và Hội đồng để có thể rút kinh nghiệm cho các công việc nghiên cứu, thiết kế hệ thống sau này.

Em xin chân thành cảm ơn!

MỤC LỤC

LỜI NÓI ĐẦU	2
LỜI CẢM ƠN.....	4
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI	10
1.1. GIỚI THIỆU CHUNG VỀ ĐỀ TÀI.....	10
1.2. TÍNH CẤP THIẾT VÀ Ý NGHĨA KHOA HỌC	11
1.2.1. Thực trạng kiểm tra PCB	11
1.2.2. Ý nghĩa khoa học và thực tiễn.....	12
1.3. MỤC TIÊU VÀ PHẠM VI NGHIÊN CỨU	12
1.3.1. Mục tiêu nghiên cứu.....	12
1.3.2. Phạm vi nghiên cứu.....	13
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	14
2.1. TỔNG QUAN VỀ PCB ESP32	14
2.1.1. Tổng quan về bảng mạch in PCB	14
2.1.2. Bo mạch ESP32 DevKit	15
2.1.3. Khảo sát các dạng lỗi PCB và lựa chọn đối tượng nghiên cứu.....	15
2.1.4. Vai trò của PCB trong hệ thống kiểm tra tự động	16
2.2. TỔNG QUAN VỀ COMPUTER VISION	16
2.2.1. Quy trình xử lý ảnh trong Computer Vision	17
2.2.2. Tiền xử lý dữ liệu và tăng cường dữ liệu ảnh	18
2.2.3. Vai trò của Computer Vision trong đề tài.....	19
2.3. MẠNG SINH ĐỐI KHÁNG VÀ WGAN-GP	19
2.3.1. Khái niệm GANs.....	19
2.3.2. Mô hình WGAN-GP.....	20
2.3.3. Nguyên lý huấn luyện WGAN-GP	21
2.3.4. Hàm mất mát và Gradient Penalty.....	21
2.3.5. Ưu điểm và hạn chế của WGAN-GP	22
2.3.6. Ứng dụng của WGAN-GP trong đề tài	22
2.4. THUẬT TOÁN YOLO	23
2.4.1. Giới thiệu thuật toán YOLO	23
2.4.2. Nguyên lý hoạt động của YOLO	23
2.4.3. YOLO Classification và YOLO Detection	24
2.4.4. Ứng dụng của YOLO trong đề tài	25
2.5. MÔI TRƯỜNG MÔ PHỎNG UNITY 3D	25
2.5.1. Giới thiệu Unity 3D	26
2.5.2. Vai trò của Unity 3D trong đề tài.....	26
2.5.3. Các thành phần chính của hệ thống mô phỏng	27

2.5.4. Quy trình hoạt động của hệ thống mô phỏng	27
2.5.5. Ưu điểm của việc sử dụng Unity 3D	28
2.6. PHẦN MỀM ROBOFLOW	28
2.7. PHẦN MỀM VISUAL STUDIO CODE	29
2.8. NỀN TẢNG GOOGLE COLAB.....	29
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG	31
3.1. THIẾT KẾ TỔNG THỂ HỆ THỐNG.....	31
3.1.1. Tổng quan quá trình xây dựng hệ thống.....	31
3.2. XÂY DỰNG BỘ DỮ LIỆU VÀ CÁC PHƯƠNG PHÁP MỞ RỘNG DỮ LIỆU.....	32
3.3. HUẤN LUYỆN MÔ HÌNH YOLO CLASSIFICATION KẾT HỢP WGAN-GP.....	33
3.3.1. Chuẩn bị bộ dữ liệu cho YOLO Classification	33
3.3.2. Huấn luyện mô hình WGAN-GP	34
3.3.3. Sinh ảnh và lọc dữ liệu sinh bởi WGAN-GP.....	37
3.4. HUẤN LUYỆN MÔ HÌNH YOLO DETECTION KẾT HỢP WGAN-GP	37
3.4.1. Chuẩn bị dữ liệu cho YOLO Detection	37
3.4.2. Huấn luyện YOLO Detection trên tập dữ liệu gốc.....	38
3.4.3. Kết hợp WGAN-GP để mở rộng dữ liệu Detection.....	38
3.4.4. Xuất mô hình và chuẩn bị tích hợp Unity	38
3.5. SO SÁNH HAI HƯỚNG VÀ LỰA CHỌN MÔ HÌNH CHÍNH	39
3.5.1. Tiêu chí so sánh	39
3.5.2. Lý do lựa chọn YOLO Detection kết hợp WGAN-GP.....	39
3.6. XÂY DỰNG HỆ THỐNG MÔ PHỎNG TRÊN UNITY 3D	40
3.6.1. Thiết kế môi trường mô phỏng	40
3.6.2. Xây dựng logic điều khiển hệ thống.....	41
3.6.3. Thiết kế quy trình kiểm tra và phân loại PCB	43
3.6.4. Thiết kế màn hình UI theo dõi và hiển thị kết quả.....	44
CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	45
4.1. KẾT QUẢ HUẤN LUYỆN WGAN-GP	45
4.2. KẾT QUẢ THỰC NGHIỆM YOLO CLASSIFICATION.....	46
4.2.1. YOLO Classification trên dữ liệu gốc	46
4.2.2. YOLO Classification sau khi bổ sung ảnh WGAN-GP	47
4.3. KẾT QUẢ THỰC NGHIỆM YOLO DETECTION	49
4.3.1. YOLO Detection trên dữ liệu gốc	49
4.3.2. YOLO Detection sau khi bổ sung ảnh WGAN-GP	50
4.4. SO SÁNH VÀ LỰA CHỌN MÔ HÌNH TRIỂN KHAI	52
4.5. KẾT QUẢ CHẠY MÔ PHỎNG TRÊN UNITY 3D	53
4.6. KẾT LUẬN CHƯƠNG.....	55

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	56
5.1. KẾT LUẬN CHUNG	56
5.1.1. Tóm tắt thành tựu của đề tài.....	56
5.1.2. Đánh giá mức độ hoàn thành mục tiêu.....	57
5.1.3. Giá trị khoa học và thực tiễn của đề tài	57
5.2. PHÂN TÍCH ƯU ĐIỂM VÀ HẠN CHẾ	58
5.2.1. Ưu điểm nổi bật của hệ thống	58
5.2.2. Hạn chế còn tồn tại.....	58
5.3. HƯỚNG PHÁT TRIỂN TRONG TƯƠNG LAI.....	59
5.4. LỜI KẾT	60
TÀI LIỆU THAM KHẢO	61
PHỤ LỤC.....	62

DANH MỤC HÌNH ẢNH

Hình 1.1. Mô hình tổng quan hệ thống kiểm tra PCB tự động	11
Hình 1.2. Quy trình kiểm tra PCB bằng Computer Vision và YOLO Detection	12
Hình 2.1. Cấu tạo cơ bản của bảng mạch in PCB.....	14
Hình 2.2. Bo mạch ESP32 DevKit và các linh kiện chính cần kiểm tra.....	15
Hình 2.3. Các trạng thái PCB được dùng để xây dựng tập dữ liệu	16
Hình 2.4. Mô hình tổng quát của hệ thống Computer Vision	17
Hình 2.5. Quy trình xử lý ảnh trong hệ thống kiểm tra PCB.....	18
Hình 2.6. Minh họa các phép biến đổi Data Augmentation trên ảnh PCB.....	19
Hình 2.7. Kiến trúc tổng quát của mạng GANs.....	20
Hình 2.8. Kiến trúc WGAN-GP sử dụng Generator và Critic.....	21
Hình 2.9. Quy trình sử dụng và lọc ảnh sinh bởi WGAN-GP	22
Hình 2.10. Nguyên lý hoạt động của mô hình YOLO	24
Hình 2.11. Ví dụ đầu ra của YOLO Classification và YOLO Detection	24
Hình 2.12. Giao diện phần mềm Unity 3D	26
Hình 2.13. Các thành phần chính trong môi trường mô phỏng Unity	27
Hình 2.14. Quy trình hoạt động của hệ thống mô phỏng Unity	28
Hình 2.15. Giao diện phần mềm Roboflow dùng để gán nhãn dữ liệu Detection	29
Hình 2.16. Giao diện Visual Studio Code dùng trong quá trình lập trình.....	29
Hình 2.17. Môi trường Google Colab dùng để huấn luyện mô hình.....	30
Hình 3.1. Nguyên lý hoạt động của hệ thống.....	31
Hình 3.2. Quy trình xây dựng dữ liệu và lựa chọn mô hình YOLO Detection.....	32
Hình 3.3. Cấu trúc thư mục dữ liệu cho YOLO Classification	33
Hình 3.4. Cấu hình huấn luyện YOLO Classification trên Google Colab	34
Hình 3.5. Kết quả theo dõi quá trình huấn luyện WGAN-GP trên Google Colab	36
Hình 3.6. Một số ảnh PCB sinh bởi WGAN-GP sau khi huấn luyện	37
Hình 3.7. Gán nhãn bounding box cho dữ liệu YOLO Detection	37
Hình 3.8. Quá trình huấn luyện YOLO Detection trên Google Colab	38
Hình 3.9. Kết quả YOLO Detection trên ảnh kiểm thử PCB.....	39
Hình 3.10. Khối kiểm tra PCB và các đối tượng trong Unity Scene	40
Hình 3.11. Khối phân loại sản phẩm và các đối tượng trong Unity Scene	41
Hình 3.12. Logic hoạt động của khối quản lý PCB	42
Hình 3.13. Logic hoạt động của khối kiểm tra PCB	42
Hình 3.14. Logic hoạt động của khối phân loại sản phẩm.....	42
Hình 3.15. Quy trình hoạt động tổng thể của hệ thống mô phỏng	43

Hình 3.16. Màn hình chờ hiển thị khi chưa chạy mô phỏng	44
Hình 3.17 Màn hình theo dõi kết quả khi ô hình đang chạy	44
Hình 4.1. Kết quả huấn luyện và ảnh sinh của WGAN-GP.....	46
Hình 4.2. Sơ đồ kết quả huấn luyện YOLO Classification trước và sau khi bổ sung WGAN-GP	48
Hình 4.3. Ma trận nhầm lẫn của YOLO Classification.....	48
Hình 4.4. Sơ đồ kết quả huấn luyện YOLO Detection trước và sau khi bổ sung WGAN-GP	51
Hình 4.6. Giao diện màn hình UI Unity khi phân loại PCB.....	54
Hình 4.7. Giao diện màn hình UI Unity thống kê số lượng PCB train.....	54

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. GIỚI THIỆU CHUNG VỀ ĐỀ TÀI

Trong dây chuyền sản xuất điện tử, PCB là phần nền để gắn linh kiện và bảo đảm thiết bị hoạt động ổn định. Những lỗi nhỏ như thiếu linh kiện, sai vị trí hoặc lỗi hàn đều có thể làm mạch không chạy đúng chức năng. Vì vậy, kiểm tra PCB sau lắp ráp là bước cần có trước khi sản phẩm được đưa sang công đoạn tiếp theo.

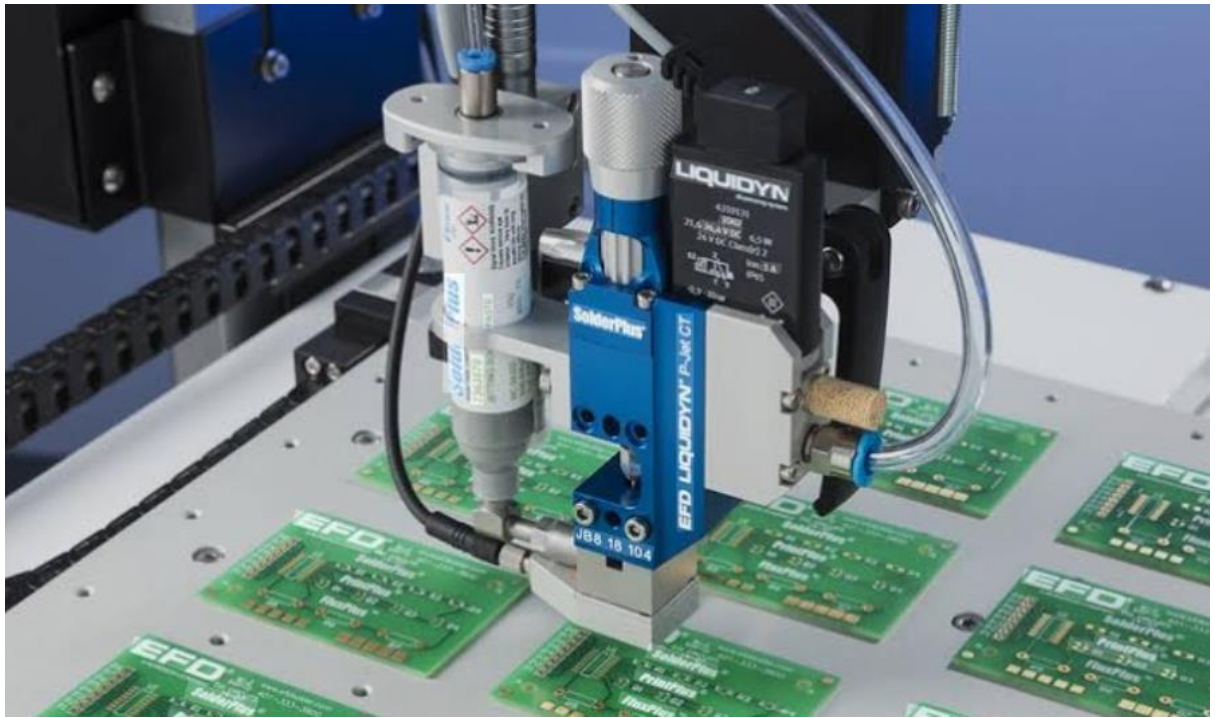
Hiện nay, việc kiểm tra PCB ở nhiều cơ sở vẫn dựa vào người vận hành hoặc các hệ thống AOI chuyên dụng. Kiểm tra thủ công dễ triển khai nhưng phụ thuộc nhiều vào kinh nghiệm và sự tập trung của người kiểm tra. Ngược lại, AOI có độ ổn định cao hơn nhưng chi phí thiết bị và bảo trì lớn. Trong điều kiện làm đồ án, việc xây dựng một mô hình kiểm tra bằng camera, mô hình nhận dạng và logic phân loại là cách phù hợp để kiểm chứng nguyên lý trước khi xét đến triển khai thực tế.

Từ yêu cầu trên, đồ án "Nghiên cứu và mô phỏng hệ thống kiểm tra chất lượng PCB tự động trong dây chuyền sản xuất điện tử sử dụng Computer Vision và AI" được triển khai. Phần mô phỏng được xây dựng trên Unity 3D, thể hiện các bước chính của một dây chuyền kiểm tra: tạo PCB, đưa PCB đến vùng kiểm tra, chụp ảnh, nhận dạng bằng YOLO Detection và phân loại sản phẩm vào khay tương ứng.

Đối tượng được chọn là bo mạch ESP32 DevKit. Bộ dữ liệu gồm mạch đạt yêu cầu và các trường hợp thiếu linh kiện: thiếu cổng USB, thiếu module ESP32, thiếu điện trở, thiếu tụ điện và thiếu vi mạch CP2102. Ở giai đoạn thực nghiệm, mỗi mẫu chủ yếu mang một lỗi thiếu linh kiện để kiểm tra khả năng nhận dạng và phân loại của hệ thống.

Trong quá trình thực hiện, đồ án khảo sát hai hướng: YOLO Classification kết hợp WGAN-GP và YOLO Detection kết hợp WGAN-GP. Classification phù hợp khi một ảnh chỉ có một trạng thái rõ ràng. Tuy nhiên, nếu một PCB thiếu nhiều linh kiện cùng lúc thì cách gán một nhãn duy nhất cho toàn ảnh sẽ khó mở rộng, vì phải tạo thêm các lớp tổ hợp. Detection hợp lý hơn trong trường hợp này, do mỗi linh kiện thiếu được xem là một đối tượng riêng với tên lớp, tọa độ và độ tin cậy.

Trọng tâm của đồ án là xây dựng một quy trình kiểm tra ở mức mô phỏng logic, gồm chuẩn bị dữ liệu, huấn luyện mô hình, tích hợp mô hình vào Unity và kiểm tra luồng phân loại sản phẩm. WGAN-GP được dùng để khảo sát khả năng bổ sung dữ liệu, còn YOLO Detection là mô hình chính dùng cho nhận dạng và hiển thị vị trí lỗi.



Hình 1.1. Mô hình tổng quan hệ thống kiểm tra PCB tự động

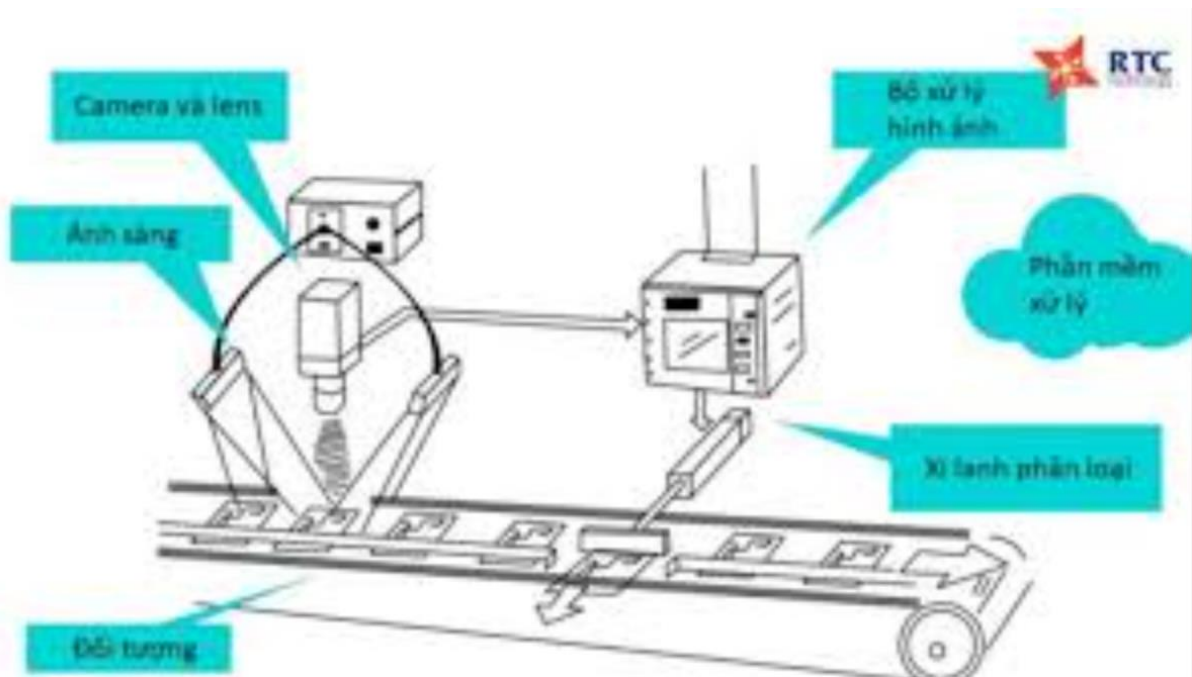
1.2. TÍNH CẤP THIẾT VÀ Ý NGHĨA KHOA HỌC

1.2.1. Thực trạng kiểm tra PCB

Trong sản xuất thực tế, ảnh lỗi thiếu linh kiện thường không có nhiều và cũng không phân bố đều giữa các loại linh kiện. Với một số lỗi hiếm, người nghiên cứu phải tạo mẫu giả lập hoặc chỉnh sửa ảnh để có dữ liệu huấn luyện. Đây là khó khăn thường gặp khi áp dụng học sâu cho bài toán kiểm tra PCB, vì mô hình cần dữ liệu đủ đa dạng mới nhận dạng ổn định.

Ngoài vấn đề dữ liệu, hệ thống kiểm tra cũng cần phản hồi nhanh để không làm chậm luồng sản phẩm. YOLO phù hợp với yêu cầu này vì có tốc độ suy luận cao và hỗ trợ trực tiếp bài toán Detection. Với Detection, kết quả không dừng ở tên lỗi mà còn có vị trí vùng lỗi trên ảnh, thuận lợi cho hiển thị và xử lý tiếp theo.

Từ các yêu cầu trên, đề án chọn hướng mô phỏng hệ thống kiểm tra PCB bằng Computer Vision và YOLO Detection. WGAN-GP được đặt ở vai trò khảo sát dữ liệu bổ sung. Ảnh sinh chỉ được dùng khi còn giữ đúng bố cục mạch và không làm sai ý nghĩa nhãn.



Hình 1.2. Quy trình kiểm tra PCB bằng Computer Vision và YOLO Detection

1.2.2. Ý nghĩa khoa học và thực tiễn

Về mặt học thuật, đồ án kết hợp ba phần: xử lý ảnh trong Computer Vision, phát hiện đối tượng bằng YOLO Detection và mô phỏng logic hệ thống trên Unity 3D. Phần WGAN-GP được dùng để khảo sát khả năng tạo thêm dữ liệu cho các lớp lỗi ít mẫu. Nhờ đó, báo cáo không chỉ đánh giá mô hình nhận dạng mà còn kiểm tra cách đưa kết quả nhận dạng vào một quy trình phân loại có thứ tự.

Về mặt thực tiễn, mô phỏng có thể dùng để minh họa nguyên lý kiểm tra PCB tự động trong học tập và nghiên cứu. Kết quả của đồ án chưa thay thế được hệ thống AOI ngoài nhà máy, nhưng có thể làm bước nền cho các nghiên cứu tiếp theo về quy trình kiểm tra, sửa lỗi và kiểm tra lại PCB.

1.3. MỤC TIÊU VÀ PHẠM VI NGHIÊN CỨU

1.3.1. Mục tiêu nghiên cứu

Mục tiêu chính của đồ án là xây dựng và đánh giá một hệ thống mô phỏng kiểm tra PCB tự động. Trong hệ thống này, YOLO Detection được dùng để phát hiện lỗi thiếu linh kiện; kết quả nhận dạng sau đó được đưa vào Unity để hiển thị và phân loại PCB.

Các nhiệm vụ chính của đề tài gồm:

- Nghiên cứu cơ sở về PCB ESP32, Computer Vision, YOLO Classification, YOLO Detection, WGAN-GP và Unity 3D.
- Xây dựng bộ dữ liệu PCB gồm mẫu đạt yêu cầu và các mẫu thiếu linh kiện; chuẩn bị dữ liệu ở cả dạng phân loại ảnh và dạng gán nhãn bounding box cho Detection.
- Huấn luyện và so sánh hai hướng YOLO Classification + WGAN-GP và YOLO Detection + WGAN-GP, từ đó lựa chọn mô hình phù hợp hơn với hệ thống kiểm tra tự động.

- Xây dựng mô phỏng logic vận hành trong Unity 3D, gồm khởi tạo PCB, di chuyển bằng script, chụp ảnh, gọi mô hình YOLO Detection, hiển thị kết quả và phân loại PCB vào các khay.

1.3.2. Phạm vi nghiên cứu

Đề tài sử dụng bo mạch ESP32 DevKit làm đối tượng nghiên cứu. Các lỗi được xét gồm PCB_OK, PCB_Missing_USB, PCB_Missing_ESP32, PCB_Missing_Resistor, PCB_Missing_Capacitor và PCB_Missing_CP2102.

Hệ thống được triển khai ở mức mô phỏng trên Unity 3D. Băng tải, cảm biến, camera, Cylinder và khay phân loại được dùng để biểu diễn quy trình làm việc. Chuyển động của PCB và thao tác phân loại được điều khiển bằng script C#, chưa mô phỏng chi tiết ma sát, lực đẩy, va chạm hoặc động học xi lanh.

Trong phạm vi đồ án, các mẫu lỗi đơn được dùng để kiểm chứng nguyên lý phát hiện và phân loại. Trường hợp PCB thiếu nhiều linh kiện cùng lúc được đặt ở hướng phát triển. Khi đó, YOLO Detection có thể trả về nhiều bounding box trên cùng một ảnh; PCB sau khi sửa sẽ được kiểm tra lại cho đến khi không còn lỗi vượt ngưỡng tin cậy.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. TỔNG QUAN VỀ PCB ESP32

PCB là phần nền để gắn và liên kết các linh kiện điện tử. Sau khi lắp ráp, chất lượng PCB ảnh hưởng trực tiếp đến khả năng hoạt động của thiết bị. Trong đồ án này, PCB được xem là đối tượng cần kiểm tra bằng ảnh, còn trạng thái của PCB là kết quả cần nhận dạng.

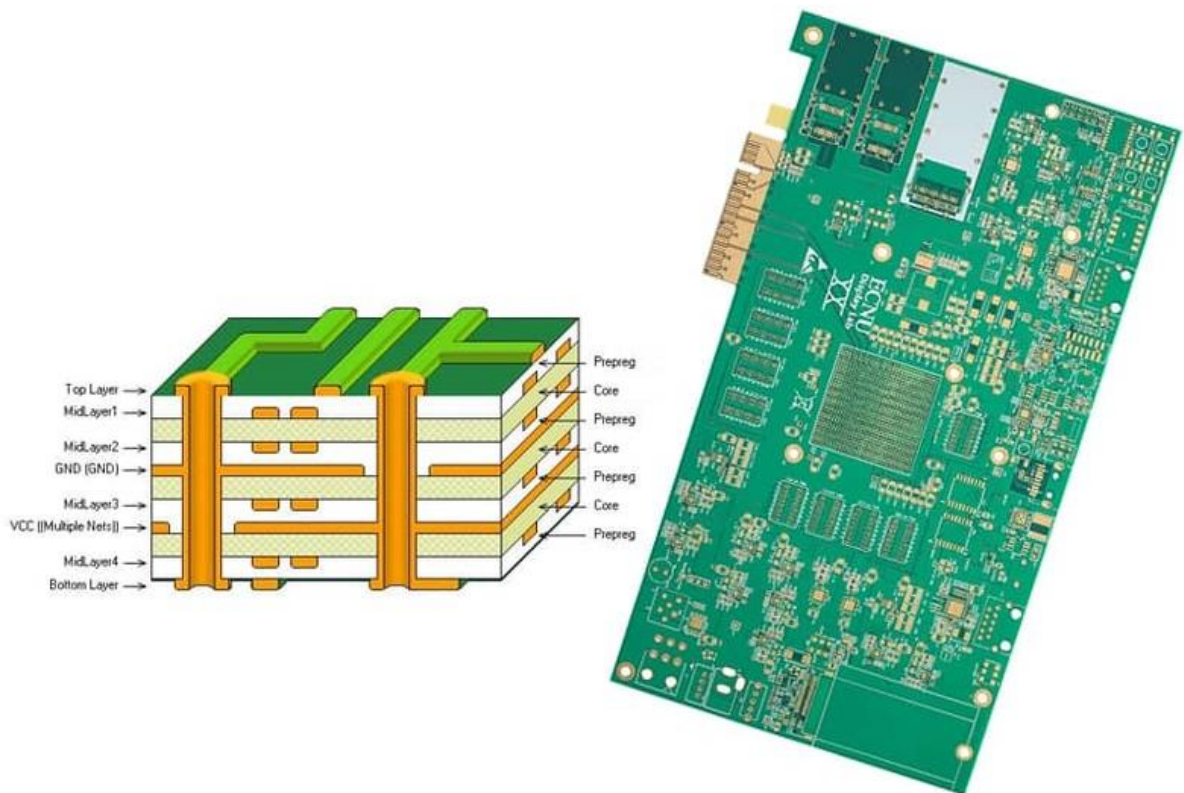
ESP32 DevKit được chọn vì bố cục linh kiện khá rõ, kích thước vừa phải và thuận lợi khi chụp từ góc nhìn trên xuống. Bo mạch này cũng phổ biến trong học tập hệ thống nhúng, nên việc tạo các trường hợp thiếu linh kiện và gán nhãn dữ liệu thuận tiện hơn.

2.1.1. Tổng quan về bảng mạch in PCB

PCB (Printed Circuit Board) là bảng mạch dùng các đường dẫn đồng để nối linh kiện theo sơ đồ thiết kế. So với đầu dây rời, PCB giúp mạch gọn hơn, ổn định hơn và phù hợp với sản xuất hàng loạt.

Tùy yêu cầu thiết kế, PCB có thể là mạch một lớp, hai lớp hoặc nhiều lớp. Trong các sản phẩm điện tử hiện nay, mật độ linh kiện tăng lên đáng kể; do đó kiểm tra bằng mắt thường khó giữ được độ ổn định khi làm việc lâu hoặc khi số lượng sản phẩm lớn.

Sau lắp ráp, PCB có thể phát sinh các lỗi như thiếu linh kiện, sai vị trí linh kiện, cầu hàn, đứt mạch hoặc ngắn mạch. Đồ án giới hạn phạm vi ở nhóm lỗi thiếu linh kiện để phù hợp với dữ liệu, cách gán nhãn và mô phỏng Unity.



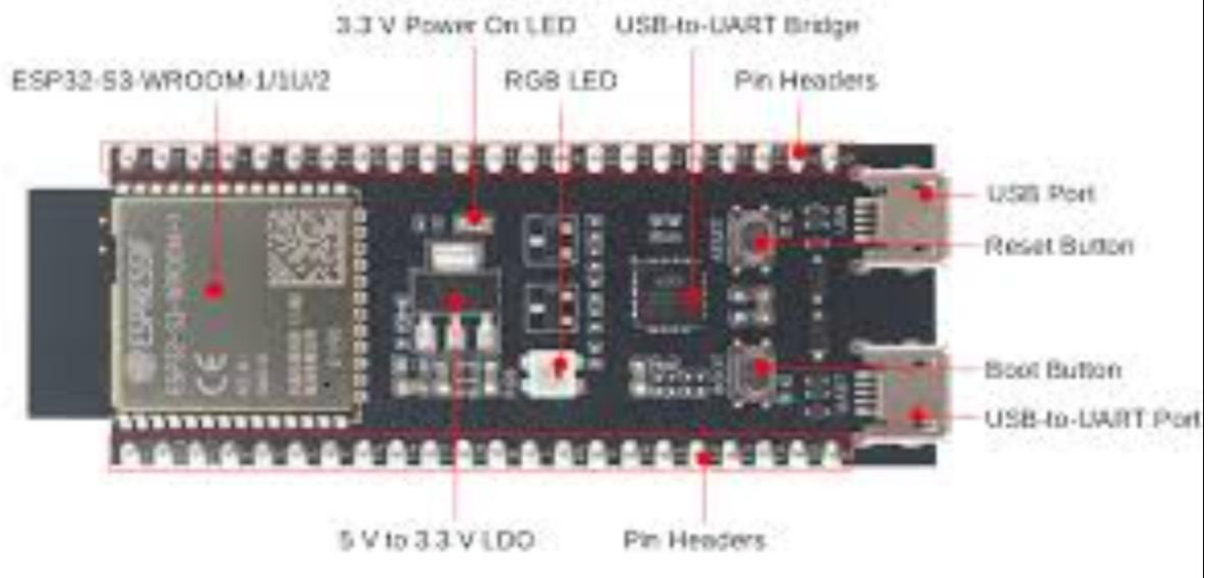
Hình 2.1. Cấu tạo cơ bản của bảng mạch in PCB

2.1.2. Bo mạch ESP32 DevKit

ESP32 DevKit là bo mạch phát triển dựa trên vi điều khiển ESP32 của Espressif Systems. Bo mạch này thường được dùng trong các ứng dụng IoT, hệ thống nhúng và các bài thực hành điều khiển nhờ có WiFi, Bluetooth và nhiều chân giao tiếp.

Trên bo mạch có một số linh kiện dễ quan sát từ góc nhìn trên xuống như cổng USB, module ESP32, IC chuyển đổi USB-UART CP2102, điện trở và tụ điện. Các linh kiện này được chọn để xây dựng các trạng thái lỗi thiếu linh kiện trong bộ dữ liệu.

Việc chọn ESP32 DevKit giúp quá trình tạo mẫu, chụp ảnh, chỉnh sửa lỗi và gán nhãn thuận lợi hơn. Bố cục bo mạch tương đối cố định cũng phù hợp với bài toán phát hiện vị trí linh kiện thiếu bằng YOLO Detection.



Hình 2.2. Bo mạch ESP32 DevKit và các linh kiện chính cần kiểm tra

2.1.3. Khảo sát các dạng lỗi PCB và lựa chọn đối tượng nghiên cứu

Trong quá trình lắp ráp PCB, lỗi có thể xuất hiện ở nhiều dạng. Tuy nhiên, không phải lỗi nào cũng phù hợp để đưa vào phạm vi đồ án. Các lỗi như cầu hàn, đứt mạch hoặc sai vị trí linh kiện cần dữ liệu ảnh chi tiết hơn và tiêu chuẩn đánh giá chặt chẽ hơn.

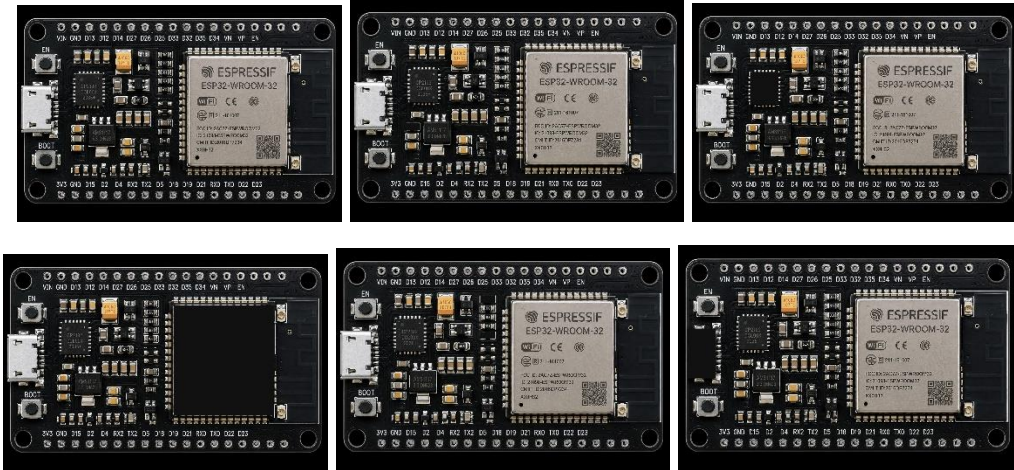
Do đó, đồ án tập trung vào lỗi thiếu linh kiện. Nhóm lỗi này quan sát được trực tiếp trên ảnh, có thể gán bounding box rõ ràng và phù hợp với cách hoạt động của YOLO Detection. Đây cũng là cơ sở để mô phỏng quá trình phân loại sản phẩm trong Unity.

Các trạng thái PCB được dùng trong bộ dữ liệu gồm:

- PCB_OK.
- PCB_Missing_USB.
- PCB_Missing_ESP32.
- PCB_Missing_Resistor.

- PCB_Missing_Capacitor.
- PCB_Missing_CP2102.

Các trạng thái trên được dùng để kiểm chứng nguyên lý kiểm tra tự động. Trong thực tế, một PCB có thể thiếu nhiều linh kiện cùng lúc; khi đó Detection có lợi thế hơn Classification vì mỗi linh kiện thiếu được phát hiện như một đối tượng riêng.



Hình 2.3. Các trạng thái PCB được dùng để xây dựng tập dữ liệu

2.1.4. Vai trò của PCB trong hệ thống kiểm tra tự động

Trong mô phỏng, PCB được tạo ở đầu băng tải và di chuyển đến vùng kiểm tra theo logic lập trình. Khi PCB đến điểm kiểm tra, camera chụp ảnh bề mặt bo mạch và ảnh này được đưa vào YOLO Detection.

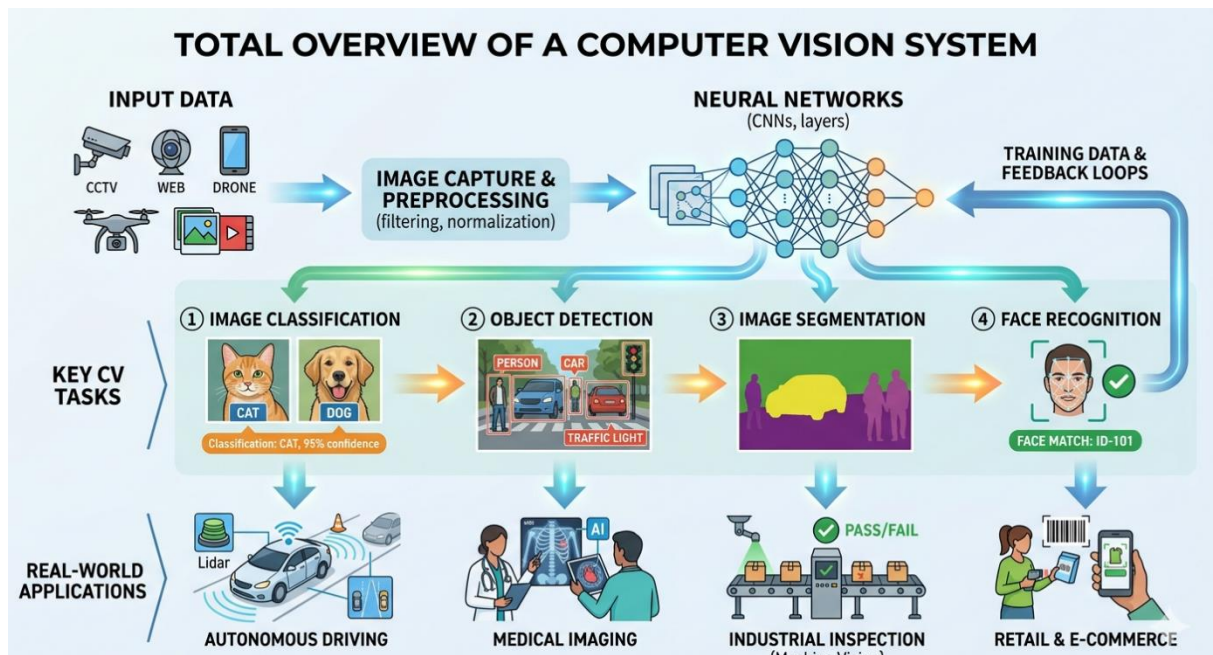
Kết quả nhận dạng được lưu kèm theo từng PCB và được khối phân loại sử dụng ở bước sau. Nhờ vậy, Unity không chỉ hiển thị ảnh và bounding box mà còn kiểm tra được luồng xử lý từ nhận dạng đến phân loại.

2.2. TỔNG QUAN VỀ COMPUTER VISION

Computer Vision nghiên cứu các phương pháp để máy tính thu nhận, xử lý và phân tích ảnh hoặc video. Trong bài toán kiểm tra PCB, Computer Vision cho phép hệ thống quan sát bo mạch bằng camera và tự động rút ra thông tin lỗi, thay vì phụ thuộc hoàn toàn vào người kiểm tra.

Trong công nghiệp, thị giác máy tính thường được dùng cho kiểm tra sản phẩm, đo kích thước, phát hiện lỗi bề mặt và phân loại linh kiện. Khi kết hợp với học sâu, mô hình có thể học đặc trưng trực tiếp từ ảnh, giảm bớt sự phụ thuộc vào các quy tắc xử lý ảnh thủ công.

Trong đồ án này, Computer Vision đóng vai trò nối giữa ảnh chụp trong Unity và mô hình YOLO Detection. Ảnh PCB sau khi chụp được đưa vào mô hình; kết quả gồm tên lỗi, độ tin cậy và vị trí bounding box được trả lại cho phần hiển thị và phân loại.



Hình 2.4. Mô hình tổng quát của hệ thống Computer Vision

2.2.1. Quy trình xử lý ảnh trong Computer Vision

Trong hệ thống kiểm tra PCB, quy trình xử lý ảnh thường đi qua các bước: thu nhận ảnh, tiền xử lý, nhận dạng bằng mô hình học sâu và trả kết quả cho khối điều khiển.

Thu nhận hình ảnh (Image Acquisition)

Camera có nhiệm vụ thu nhận ảnh PCB khi sản phẩm đến vùng kiểm tra. Trong đồ án, camera được đặt theo góc nhìn từ trên xuống để toàn bộ bề mặt bo mạch nằm trong vùng ảnh và vị trí các linh kiện ít bị che khuất.

Tiền xử lý ảnh (Image Pre-processing)

Ảnh đầu vào cần được đưa về đúng kích thước và định dạng trước khi đưa vào mô hình. Các bước tiền xử lý giúp giảm sai khác giữa các ảnh, nhất là khi ánh sáng, vị trí hoặc kích thước ảnh thay đổi.

Một số kỹ thuật tiền xử lý thường được sử dụng gồm:

- Chuẩn hóa kích thước ảnh.
- Điều chỉnh độ sáng và độ tương phản.
- Giảm nhiễu bằng các bộ lọc thích hợp.
- Chuẩn hóa dữ liệu đầu vào.

Các bước này không làm thay đổi bản chất lỗi trên PCB nhưng giúp mô hình học ổn định hơn trong quá trình huấn luyện.

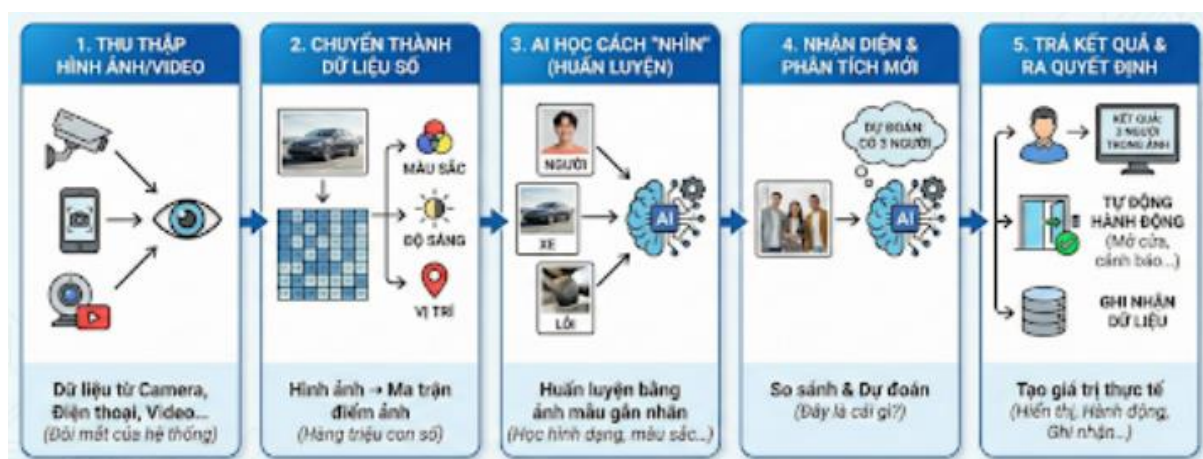
Trích xuất đặc trưng và nhận dạng

Sau bước tiền xử lý, YOLO Detection trích xuất đặc trưng từ ảnh và dự đoán các vùng có lỗi thiếu linh kiện. Mỗi kết quả gồm tên lớp, độ tin cậy và tọa độ bounding box.

Classification chỉ trả về nhãn cho toàn ảnh, trong khi Detection cho biết vị trí cụ thể của linh kiện thiếu. Vì vậy, Detection phù hợp hơn khi hệ thống cần mở rộng sang trường hợp nhiều lỗi trên cùng một PCB.

Hiển thị kết quả

Sau khi nhận dạng, kết quả được đưa về Unity để hiển thị trên giao diện. Hệ thống đồng thời lưu trạng thái PCB để phục vụ bước phân loại ở cuối băng tải mô phỏng.



Hình 2.5. Quy trình xử lý ảnh trong hệ thống kiểm tra PCB

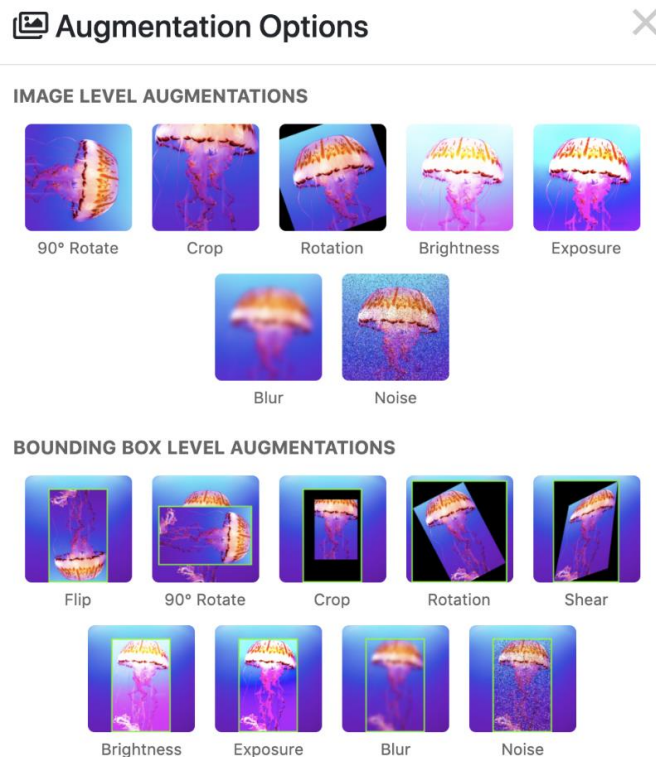
2.2.2. Tiền xử lý dữ liệu và tăng cường dữ liệu ảnh

Chất lượng dữ liệu ảnh ảnh hưởng trực tiếp đến kết quả huấn luyện. Nếu dữ liệu ít hoặc quá giống nhau, mô hình dễ học theo nền ảnh hoặc bố cục cố định thay vì học đúng đặc trưng của lỗi.

Các ảnh PCB trong đề tài được xây dựng với cùng góc nhìn từ trên xuống và được chuẩn hóa kích thước trước khi huấn luyện. Sau đó, Data Augmentation được dùng để tạo thêm biến thể ảnh.

Các phép biến đổi gồm thay đổi độ sáng, độ tương phản, dịch chuyển, xoay nhẹ và thu phóng. Các phép này được chọn ở mức vừa phải để không làm sai vị trí linh kiện hoặc làm mất ý nghĩa nhãn bounding box.

Data Augmentation làm dữ liệu đa dạng hơn, giảm nguy cơ quá khớp và giúp mô hình ổn định hơn khi gặp ảnh kiểm thử.



Hình 2.6. Minh họa các phép biến đổi Data Augmentation trên ảnh PCB

2.2.3. Vai trò của Computer Vision trong đề tài

Trong hệ thống, Computer Vision đảm nhiệm phần nhận dạng ảnh, còn Unity đảm nhiệm phần mô phỏng logic vận hành. Khi PCB đến điểm kiểm tra, ảnh được chụp và gửi đến YOLO Detection. Kết quả sau đó được trả về để hiển thị và xác định khay phân loại.

Cách tổ chức này cho phép kiểm tra quá trình phối hợp giữa mô hình nhận dạng và các script điều khiển. Trọng tâm của mô phỏng là luồng xử lý, không phải mô phỏng chi tiết cơ học của băng tải hoặc xi lanh.

2.3. MẠNG SINH ĐỐI KHÁNG VÀ WGAN-GP

Trong học sâu, dữ liệu huấn luyện quyết định nhiều đến khả năng tổng quát của mô hình. Với PCB lỗi, dữ liệu thực tế thường ít và không đều giữa các lớp. Vì vậy, ngoài Data Augmentation truyền thống, đề án khảo sát thêm WGAN-GP như một hướng tạo dữ liệu bổ sung.

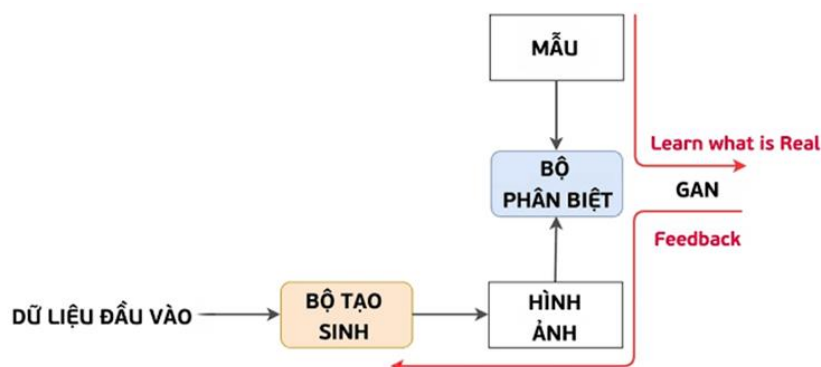
Tuy nhiên, ảnh PCB đòi hỏi cấu trúc khá chính xác. Nếu ảnh sinh bị mờ, méo hoặc sai vị trí linh kiện thì không nên đưa thẳng vào tập huấn luyện. Vì lý do đó, WGAN-GP chỉ được xem là phương pháp hỗ trợ và phải có bước lọc ảnh trước khi sử dụng.

2.3.1. Khái niệm GANs

GANs gồm hai mạng học sâu: Generator và Discriminator. Generator sinh ảnh mới từ vector nhiễu, còn Discriminator đánh giá ảnh sinh có giống ảnh thật hay không.

Quá trình huấn luyện diễn ra theo cơ chế đối kháng: Generator cố tạo ảnh giống dữ liệu thật hơn, còn Discriminator cố phân biệt ảnh thật với ảnh sinh. Khi cân bằng tương đối, Generator có thể tạo ra những ảnh mang đặc điểm gần với tập dữ liệu ban đầu.

Với ảnh PCB, sinh dữ liệu khó hơn so với nhiều loại ảnh thông thường. Bố cục linh kiện, đường mạch và vùng thiếu linh kiện phải giữ được ý nghĩa hình học. Vì vậy, ảnh sinh cần được kiểm tra trước khi dùng làm dữ liệu huấn luyện.



CÁCH THỨC HOẠT ĐỘNG CỦA MẠNG GAN

Hình 2.7. Kiến trúc tổng quát của mạng GANs

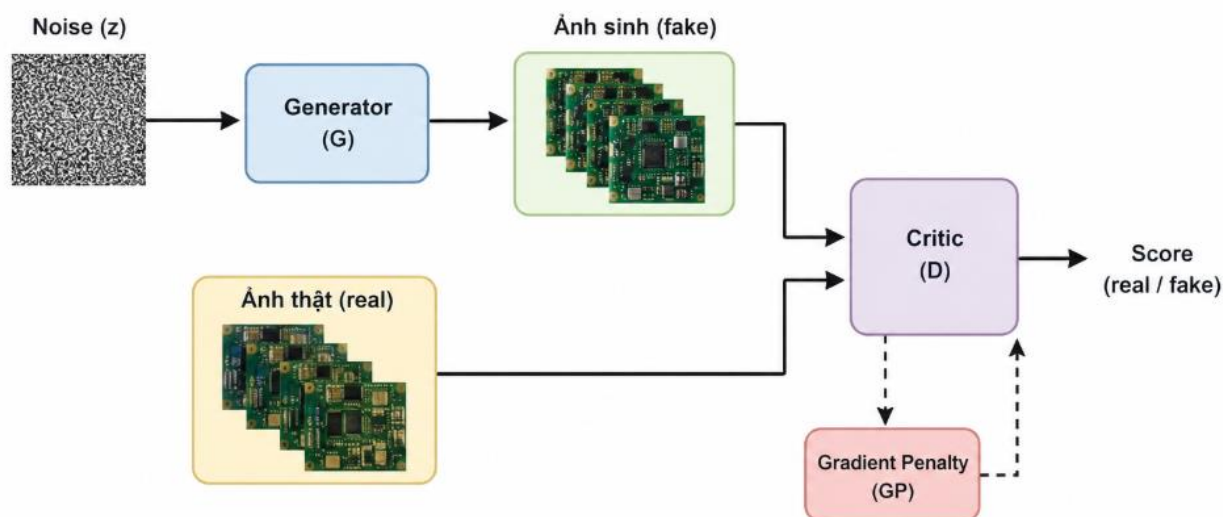
2.3.2. Mô hình WGAN-GP

WGAN-GP là biến thể của GAN dùng khoảng cách Wasserstein và thành phần Gradient Penalty để ổn định quá trình huấn luyện. Thay vì dùng Discriminator phân loại thật - giả theo xác suất, WGAN-GP dùng Critic để đánh giá mức độ khác nhau giữa phân bố ảnh thật và ảnh sinh.

Trong đồ án, WGAN-GP được dùng để thử mở rộng dữ liệu cho các trạng thái PCB đã xây dựng. Ảnh sinh được quan sát và lọc trước khi đưa vào huấn luyện YOLO. Những ảnh quá mờ, sai bố cục hoặc làm sai vùng linh kiện sẽ bị loại để tránh làm nhiễu dữ liệu.

So với GAN cơ bản, WGAN-GP thường dễ huấn luyện ổn định hơn. Tuy vậy, chất lượng ảnh sinh vẫn phụ thuộc vào dữ liệu gốc, số lượng mẫu và cách chọn tham số.

Kiến trúc WGAN-GP trong đồ án gồm Generator và Critic. Generator nhận vector nhiễu và tạo ảnh PCB mới. Critic nhận ảnh thật và ảnh sinh để đánh giá sự khác biệt giữa hai phân bố dữ liệu.



Hình 2.8. Kiến trúc WGAN-GP sử dụng Generator và Critic

Generator sử dụng các lớp tích chập chuyển vị để tăng dần kích thước đặc trưng và tạo ảnh RGB. Critic sử dụng các lớp tích chập để trích xuất đặc trưng ảnh. Hai mạng được cập nhật luân phiên trong quá trình huấn luyện.

Khác với GAN truyền thống, Critic của WGAN-GP không xuất ra xác suất thật - giả mà cho điểm đánh giá theo khoảng cách Wasserstein. Thành phần Gradient Penalty giúp ràng buộc quá trình học và hạn chế hiện tượng huấn luyện mất ổn định.

Trong đồ án, WGAN-GP được triển khai ở mức khảo sát. Mục tiêu là xem mô hình có thể tạo thêm ảnh hỗ trợ huấn luyện hay không; ảnh sinh không được xem là nguồn dữ liệu chính.

2.3.3. Nguyên lý huấn luyện WGAN-GP

Quá trình trên được lặp lại qua nhiều epoch. Chất lượng ảnh sinh phụ thuộc vào số lượng ảnh gốc, độ đa dạng của dữ liệu và lựa chọn tham số huấn luyện.

Với tập ảnh PCB còn ít, WGAN-GP không thể thay thế dữ liệu thật mà chỉ khảo sát khả năng tạo thêm mẫu hỗ trợ. Vì vậy, ảnh sinh cần được lọc trước khi đưa sang bước huấn luyện YOLO.

2.3.4. Hàm mất mát và Gradient Penalty

Trong WGAN-GP, Critic đánh giá sự khác biệt giữa phân bố ảnh thật và ảnh sinh, còn Generator cố tạo ảnh để làm giảm sự khác biệt đó.

Gradient Penalty được thêm vào để giúp Critic không thay đổi quá mạnh, từ đó làm quá trình huấn luyện ổn định hơn so với GAN truyền thống.

Hàm mất mát của WGAN-GP được dùng để điều chỉnh cả Generator và Critic. Critic có đánh giá rõ sự khác biệt giữa ảnh thật và ảnh sinh, còn Generator cố giảm sự khác biệt đó.

Gradient Penalty giúp quá trình huấn luyện bớt dao động và hạn chế tình trạng một mạng học quá nhanh so với mạng còn lại. Đây là lý do đề án dùng WGAN-GP thay vì GAN cơ bản.

Dù vậy, loss của GAN không phản ánh đầy đủ chất lượng ảnh khi quan sát trực quan. Vì thế, ngoài theo dõi loss, ảnh sinh vẫn phải được kiểm tra và loại bỏ các mẫu không đạt.

2.3.5. Ưu điểm và hạn chế của WGAN-GP

WGAN-GP có thể hỗ trợ tạo thêm dữ liệu khi ảnh lỗi còn ít. Tuy nhiên, mô hình cần thời gian huấn luyện, tài nguyên tính toán và dữ liệu gốc đủ đa dạng. Với ảnh PCB, nếu dữ liệu đầu vào quá hẹp, ảnh sinh dễ bị mờ, méo hoặc sai chi tiết linh kiện.

2.3.6. Ứng dụng của WGAN-GP trong đề tài

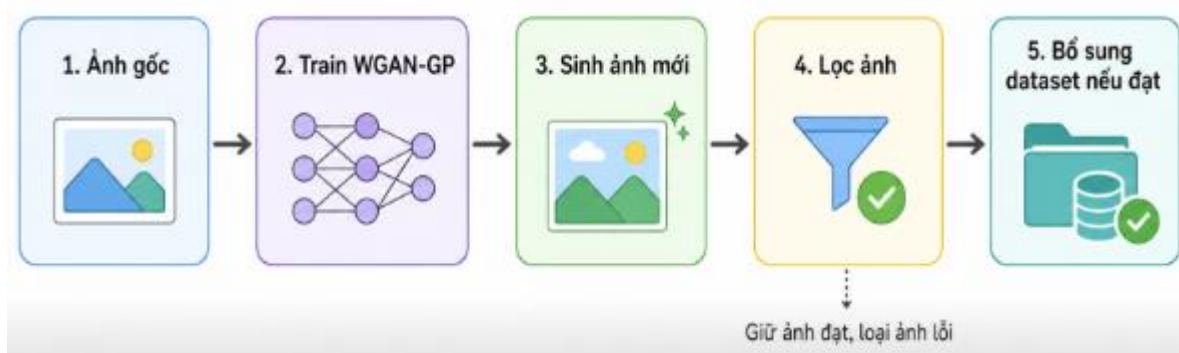
Trong đề án, WGAN-GP được dùng để thử mở rộng dữ liệu cho các lớp PCB thiếu linh kiện. Ảnh sinh chỉ được bổ sung khi còn giữ đúng bố cục mạch và không làm sai ý nghĩa nhãn.

Hạn chế chính là mô hình cần nhiều thời gian huấn luyện và phụ thuộc vào dữ liệu gốc. Khi tập ảnh ban đầu còn hẹp, Generator dễ tạo ra ảnh mờ, méo hoặc thiếu chi tiết. Vì vậy, WGAN-GP không được xem là nguồn dữ liệu chính trong đề án.

Trong đề án này, WGAN-GP giữ vai trò hỗ trợ. Kết quả cuối cùng vẫn được đánh giá chủ yếu qua khả năng nhận dạng của YOLO Detection và khả năng vận hành đúng luồng của hệ thống Unity.

WGAN-GP không dùng để nhận dạng PCB. Mô hình này chỉ được khảo sát để tạo thêm ảnh phục vụ huấn luyện khi dữ liệu lỗi còn ít.

Quy trình dùng WGAN-GP gồm chuẩn bị ảnh đầu vào, huấn luyện Generator và Critic, sinh ảnh mới, lọc ảnh không đạt, sau đó chỉ bổ sung các ảnh phù hợp vào tập dữ liệu. Với bài toán Detection, ảnh sinh còn phải giữ đúng vị trí linh kiện để nhãn bounding box vẫn có ý nghĩa.



Hình 2.9. Quy trình sử dụng và lọc ảnh sinh bởi WGAN-GP

Cách dùng này phản ánh đúng vai trò của WGAN-GP: hỗ trợ dữ liệu, không thay thế dữ liệu thật và không phải thành phần quyết định hoạt động của hệ thống.

2.4. THUẬT TOÁN YOLO

2.4.1. Giới thiệu thuật toán YOLO

YOLO (You Only Look Once) là mô hình học sâu được Joseph Redmon và cộng sự giới thiệu để giải quyết bài toán phát hiện đối tượng trong ảnh. Thay vì tách riêng nhiều bước như tìm vùng quan tâm, trích xuất đặc trưng và phân loại, YOLO thực hiện dự đoán trong một lần truyền ảnh qua mạng nơ-ron.

Nhờ cơ chế xử lý một lần, YOLO có thời gian suy luận ngắn và vẫn giữ được độ chính xác tốt khi dữ liệu huấn luyện phù hợp. Đây là lý do YOLO được dùng nhiều trong các bài toán cần xử lý nhanh.

Qua các phiên bản, YOLO được cải tiến về tốc độ, độ chính xác và khả năng hỗ trợ nhiều tác vụ thị giác máy tính. Các phiên bản gần đây như YOLOv8 và YOLO11 có sẵn các chế độ huấn luyện cho Classification, Detection và một số tác vụ khác.

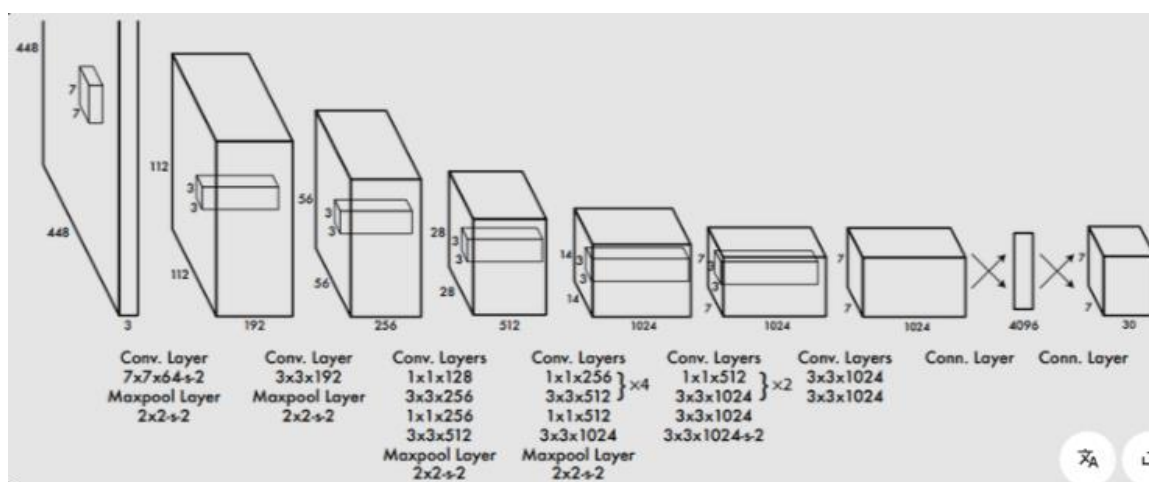
Trong đồ án, YOLO11 của Ultralytics được dùng để huấn luyện cả hai hướng Classification và Detection. Việc dùng cùng một hệ sinh thái giúp quá trình chuẩn bị dữ liệu, huấn luyện và so sánh kết quả thuận tiện hơn.

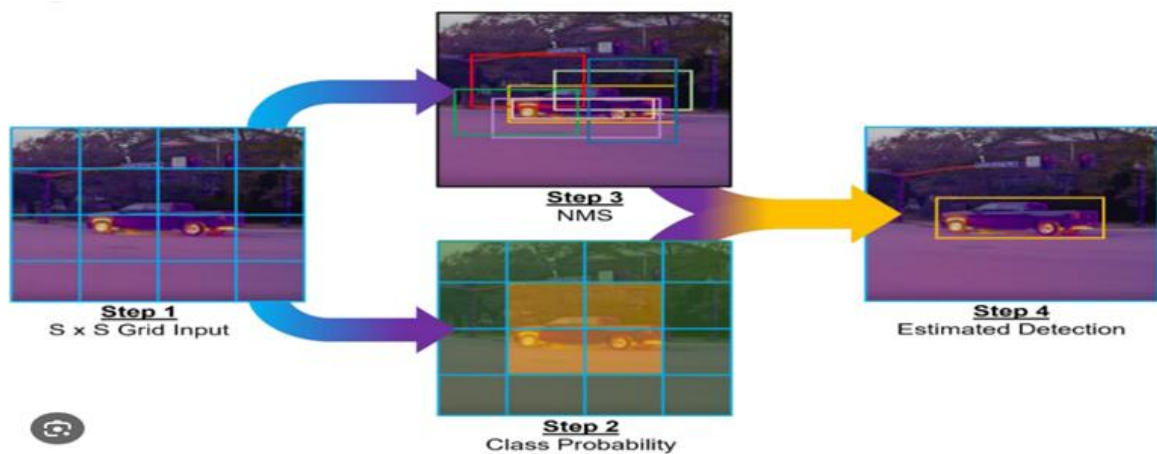
2.4.2. Nguyên lý hoạt động của YOLO

YOLO xử lý toàn bộ ảnh đầu vào trong một lần suy luận. Khi ảnh đi qua mạng, mô hình trích xuất các đặc trưng như hình dạng, màu sắc, kết cấu và quan hệ giữa các vùng ảnh để đưa ra dự đoán.

Tùy bài toán, đầu ra của YOLO có dạng khác nhau. Với Detection, mô hình trả về vị trí đối tượng, tên lớp và độ tin cậy. Với Classification, mô hình trả về xác suất của các lớp và chọn lớp có xác suất cao nhất cho toàn ảnh.

Vì quá trình xử lý gọn, YOLO thường nhanh hơn các phương pháp phát hiện hai giai đoạn. Đặc điểm này phù hợp với hệ thống kiểm tra trên dây chuyền, nơi thời gian phản hồi của mô hình ảnh hưởng trực tiếp đến nhịp vận hành.





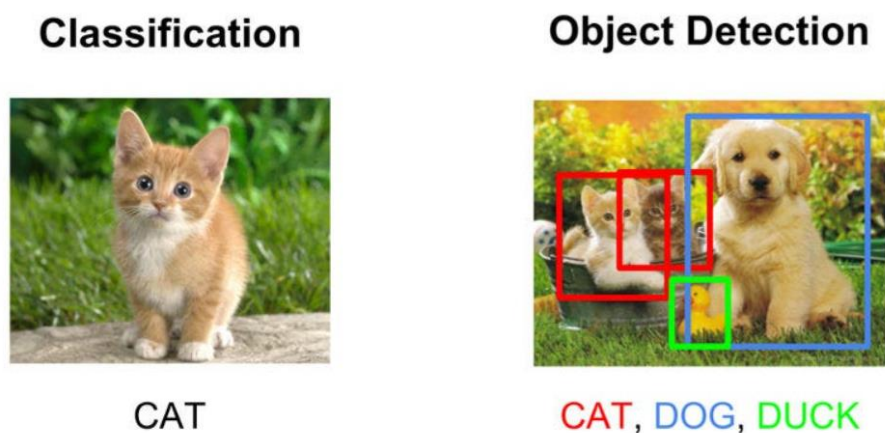
Hình 2.10. Nguyên lý hoạt động của mô hình YOLO

2.4.3. YOLO Classification và YOLO Detection

Trong đồ án, YOLO được khảo sát theo hai hướng. Hướng thứ nhất là YOLO Classification: toàn bộ ảnh PCB được đưa vào mô hình và đầu ra là một nhãn trạng thái của bo mạch. Dữ liệu của hướng này dễ chuẩn bị vì không cần gán tọa độ, phù hợp với giai đoạn thử nghiệm khi mỗi ảnh chỉ có một trạng thái rõ ràng.

Hạn chế của Classification xuất hiện khi một PCB có nhiều linh kiện thiếu cùng lúc. Ví dụ, nếu một mẫu vừa thiếu USB vừa thiếu CP2102 thì tập lớp lỗi đơn không còn đủ để mô tả. Nếu tạo thêm các lớp tổ hợp như PCB_Missing_USB_CP2102 hoặc PCB_Missing_USB_ESP32, số lớp sẽ tăng nhanh và việc thu thập dữ liệu cho từng tổ hợp trở nên khó quản lý.

YOLO Detection xử lý vấn đề trên bằng cách xem mỗi linh kiện thiếu là một đối tượng cần phát hiện. Một ảnh có thể không có lỗi, có một lỗi hoặc có nhiều lỗi; mỗi lỗi có class, bounding box và confidence riêng. Đây là lý do Detection được chọn làm hướng chính cho hệ thống mô phỏng.



Hình 2.11. Ví dụ đầu ra của YOLO Classification và YOLO Detection

Có nhiều mô hình có thể dùng cho nhận dạng ảnh như AlexNet, VGG, ResNet, EfficientNet hoặc Vision Transformer. Trong đề án này, YOLO được chọn vì phù hợp hơn với yêu cầu vừa nhận dạng lỗi vừa tích hợp vào quy trình kiểm tra mô phỏng.

Về tốc độ, YOLO xử lý ảnh trong một lần suy luận nên phù hợp với bài toán kiểm tra cần phản hồi nhanh.

Về độ chính xác, YOLO có khả năng học đặc trưng tốt khi dữ liệu huấn luyện được chuẩn bị đúng, đặc biệt với các lỗi có vị trí tương đối ổn định trên PCB.

Ngoài ra, YOLO11 hỗ trợ cả Classification và Detection trong cùng một framework, thuận lợi cho việc so sánh hai hướng trên cùng bộ dữ liệu.

Kết quả Detection cũng có thể xuất sang ONNX để tích hợp với Unity. Đầu ra gồm tên lỗi, độ tin cậy và tọa độ vùng lỗi, phù hợp với giao diện hiển thị bounding box và logic phân loại sản phẩm.

2.4.4. Ứng dụng của YOLO trong đề tài

Trong đề án, YOLO Classification được dùng để khảo sát khả năng phân loại trạng thái toàn ảnh. YOLO Detection được dùng làm hướng chính vì đáp ứng tốt hơn yêu cầu khoanh vùng lỗi và mở rộng sang trường hợp nhiều lỗi trên cùng một PCB.

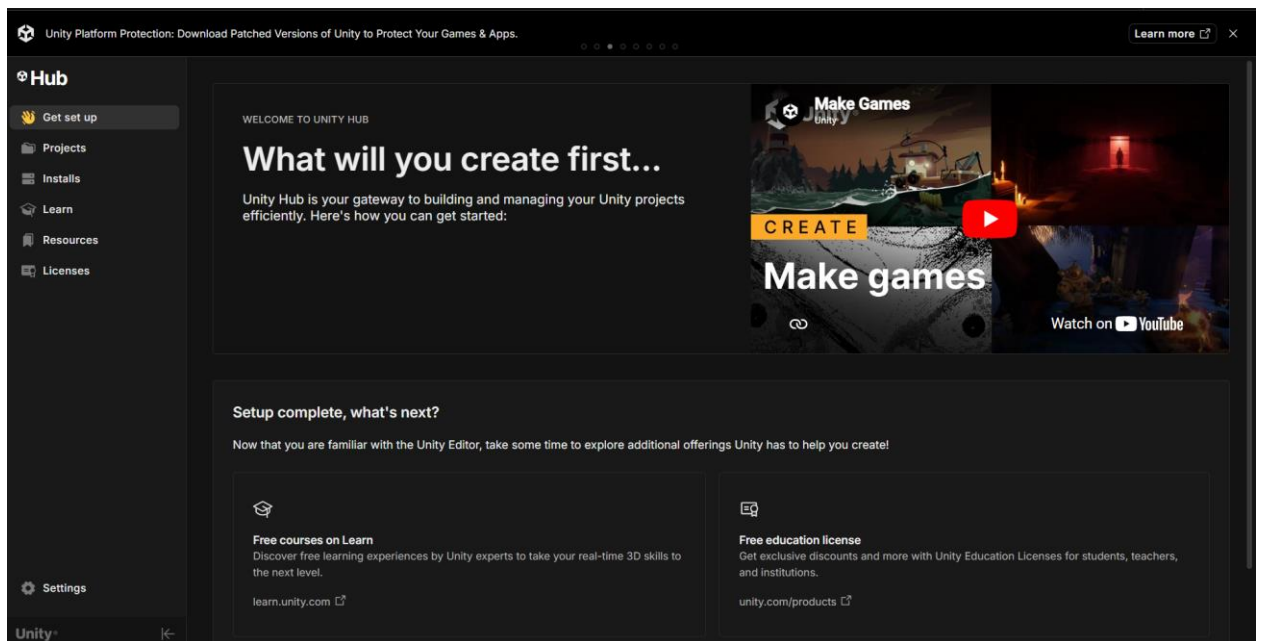
Sau khi mô hình Detection nhận dạng, kết quả được gửi về Unity để hiển thị trên ảnh chụp và lưu vào thông tin của PCB. Trạng thái này được khối phân loại đọc lại để đưa PCB vào khay tương ứng.

Việc dùng YOLO giúp liên kết phần nhận dạng ảnh với phần mô phỏng hệ thống. Kết quả không chỉ dừng ở dự đoán trên ảnh tĩnh mà được đưa vào quy trình kiểm tra - phân loại có thứ tự.

2.5. MÔI TRƯỜNG MÔ PHỎNG UNITY 3D

Unity 3D được dùng để dựng môi trường mô phỏng và kiểm tra luồng vận hành của hệ thống. Trong đề án này, Unity không mô phỏng vật lý chi tiết của băng tải hay xi lanh, mà chủ yếu trực quan hóa dây chuyền và đồng bộ các bước xử lý bằng script C#.

Cách làm này phù hợp với phạm vi đề án, vì mục tiêu chính là kiểm chứng sự phối hợp giữa camera, YOLO Detection và logic phân loại PCB. Các thông số cơ học như lực, ma sát, va chạm và động học cơ cấu được đưa vào hướng phát triển sau.



Hình 2.12. Giao diện phần mềm Unity 3D

2.5.1. Giới thiệu Unity 3D

Unity 3D là nền tảng phát triển ứng dụng thời gian thực, hỗ trợ xây dựng môi trường 3D, giao diện hiển thị và lập trình điều khiển bằng C#. Nhờ đó, người dùng có thể tạo các đối tượng như băng tải, camera, PCB, khay chứa và lập trình luồng hoạt động của chúng.

Trong đồ án, các đối tượng 3D được tạo chủ yếu để biểu diễn trực quan. Chuyển động của PCB, thời điểm chụp ảnh và thao tác phân loại được điều khiển bằng script, giúp quá trình thử nghiệm ổn định và dễ kiểm tra lỗi logic.

Unity cũng thuận tiện cho việc hiển thị ảnh camera, kết quả nhận dạng và bounding box của mô hình YOLO Detection trên giao diện mô phỏng.

2.5.2. Vai trò của Unity 3D trong đề tài

Trong đồ án, Unity đóng vai trò là môi trường mô phỏng logic vận hành của hệ thống kiểm tra PCB.

Môi trường mô phỏng gồm các thành phần: băng tải, PCB, điểm kiểm tra, camera, cảm biến vị trí, giao diện hiển thị kết quả và khối phân loại. Các thành phần này được liên kết bằng script C# để tạo thành một quy trình kiểm tra có thứ tự.

Khi PCB đến vùng kiểm tra, script điều khiển dừng hoặc giữ PCB tại vị trí chụp, camera thu ảnh và mô hình YOLO Detection thực hiện suy luận. Kết quả nhận dạng được lưu lại và dừng ở bước phân loại.

Như vậy, Unity giúp kiểm chứng việc truyền dữ liệu giữa các khối trong hệ thống: từ PCB, camera, mô hình nhận dạng đến cơ cấu phân loại mô phỏng.

2.5.3. Các thành phần chính của hệ thống mô phỏng

Hệ thống Unity được xây dựng từ các đối tượng có vai trò rõ ràng trong quy trình kiểm tra PCB.

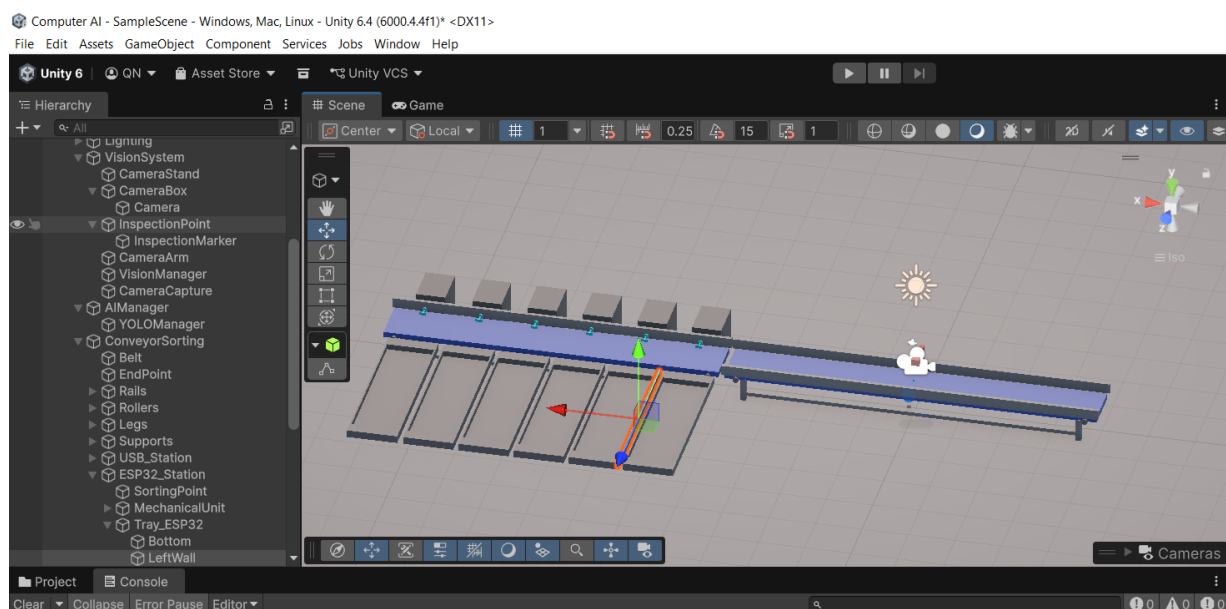
Băng tải được dựng để thể hiện hướng di chuyển của PCB trong dây chuyền. Chuyển động của PCB không dựa trên mô phỏng ma sát hay lực kéo mà được điều khiển bằng script để đồng bộ với quá trình chụp ảnh và nhận dạng.

PCB là sản phẩm cần kiểm tra. Mỗi PCB có thông tin trạng thái riêng, được dùng để đối chiếu với kết quả dự đoán của mô hình và phục vụ đánh giá vận hành.

Cảm biến vị trí được dùng như một mốc logic để xác định thời điểm PCB đi vào vùng kiểm tra. Khi PCB đến mốc này, hệ thống kích hoạt bước chụp ảnh.

Camera có nhiệm vụ chụp ảnh PCB từ góc nhìn trên xuống. Ảnh này được đưa sang mô hình YOLO Detection để nhận dạng lỗi thiếu linh kiện.

Giao diện hiển thị kết quả gồm ảnh chụp, tên lỗi, độ tin cậy và bounding box nếu có lỗi được phát hiện.



Hình 2.13. Các thành phần chính trong môi trường mô phỏng Unity

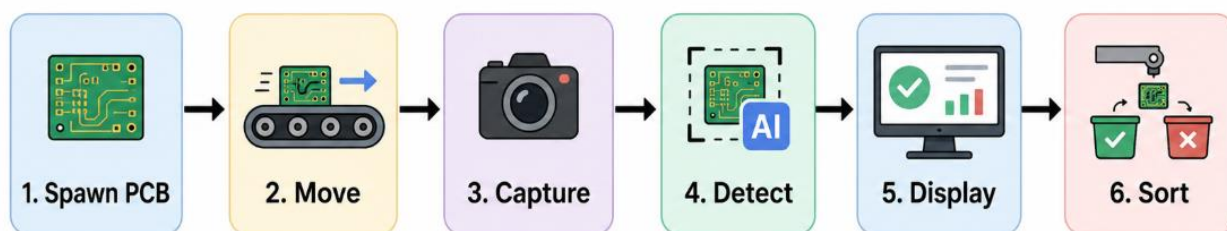
2.5.4. Quy trình hoạt động của hệ thống mô phỏng

Quy trình hoạt động của hệ thống được lập trình theo từng bước rõ ràng. PCB được khởi tạo, di chuyển đến điểm kiểm tra, được chụp ảnh, nhận dạng và sau đó đi đến khu vực phân loại.

Ở bước nhận dạng, mô hình YOLO Detection trả về danh sách lỗi phát hiện được. Với PCB_OK, mô hình không nên phát hiện lỗi vượt ngưỡng tin cậy. Với PCB lỗi, mô hình cần phát hiện đúng loại linh kiện thiếu.

Kết quả này được lưu vào dữ liệu của PCB. Khi PCB đến khối phân loại, script đọc trạng thái đã lưu và đưa PCB vào khay tương ứng.

Quy trình trên kiểm tra tính đúng đắn của luồng điều khiển và khả năng tích hợp AI vào mô phỏng. Đồ án không đánh giá các tham số vật lý của băng tải hoặc xi lanh.



Hình 2.14. Quy trình hoạt động của hệ thống mô phỏng Unity

2.5.5. Ưu điểm của việc sử dụng Unity 3D

Việc dùng Unity giúp quá trình thử nghiệm thuận tiện hơn vì có thể thay đổi tốc độ di chuyển, số lượng PCB, vị trí camera và cách hiển thị kết quả mà không cần phần cứng thật.

Môi trường mô phỏng giúp giảm chi phí thử nghiệm ban đầu và hạn chế rủi ro khi chưa có dây chuyền thực tế.

Các khối chức năng trong hệ thống có thể chỉnh sửa độc lập, ví dụ thay đổi mô hình YOLO, thay đổi logic phân loại hoặc thay đổi tập ảnh PCB.

Unity cũng giúp trực quan hóa toàn bộ quy trình, từ lúc PCB được tạo ra đến khi có kết quả nhận dạng và được đưa vào khay phân loại.

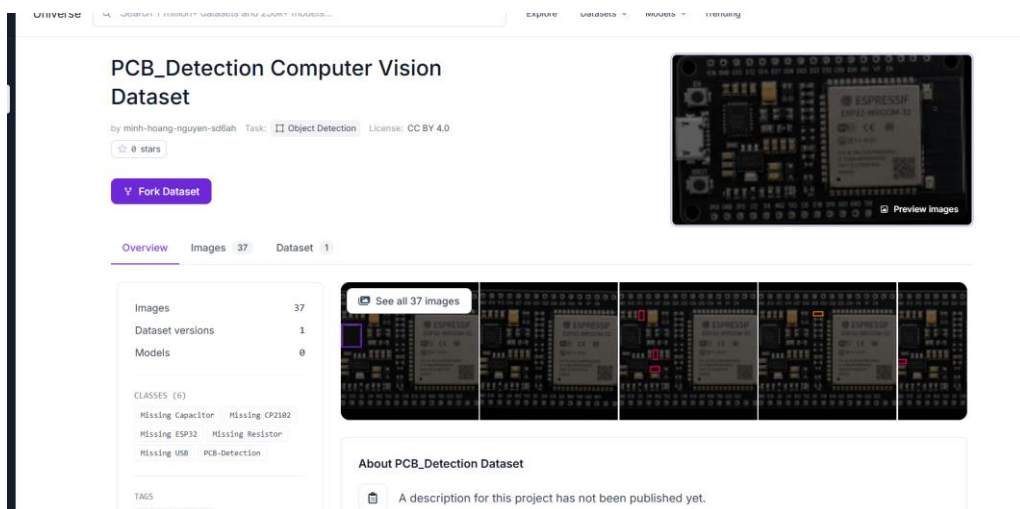
Nhờ đó, mô phỏng Unity trong đồ án đóng vai trò kiểm chứng logic hệ thống trước khi xem xét các bước triển khai thực tế phức tạp hơn.

2.6. PHẦN MỀM ROBOFLOW

Trong bài toán Detection, dữ liệu cần được gán nhãn vị trí đối tượng. Quá trình này nếu làm thủ công hoàn toàn sẽ mất thời gian và dễ sai định dạng.

Roboflow được dùng để quản lý ảnh, gán nhãn bounding box, chia tập Train/Validation/Test và xuất dữ liệu theo định dạng YOLO. Công cụ này giúp quá trình chuẩn bị dữ liệu có tổ chức hơn.

Trong đề tài, Roboflow hỗ trợ chính cho hướng YOLO Detection. Các ảnh PCB sau khi gán nhãn được xuất sang định dạng phù hợp để huấn luyện trên Google Colab.

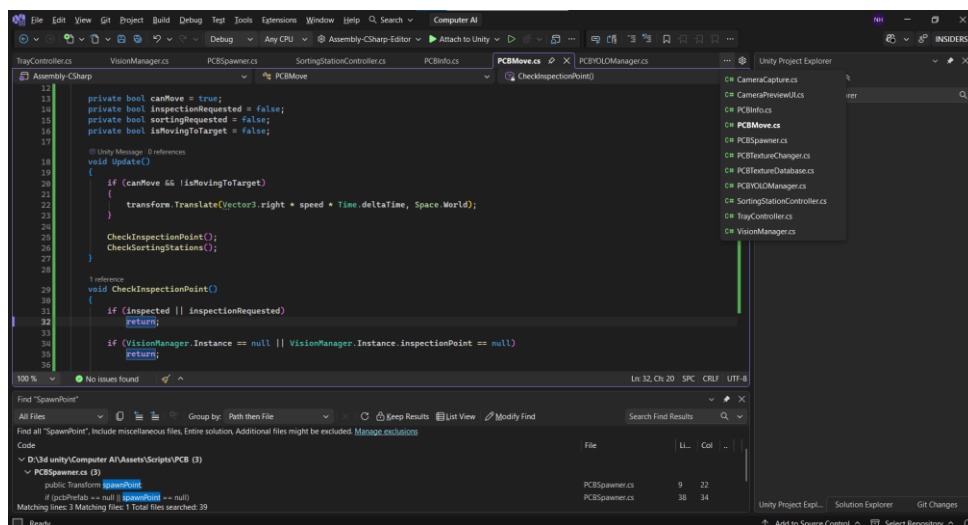


Hình 2.15. Giao diện phần mềm Roboflow dùng để gắn nhãn dữ liệu Detection

2.7. PHẦN MỀM VISUAL STUDIO CODE

Visual Studio Code được dùng để viết và chỉnh sửa mã nguồn Python, đồng thời hỗ trợ chỉnh sửa các script C# phục vụ mô phỏng Unity.

Trong quá trình thực hiện, VS Code hỗ trợ các bước xử lý dữ liệu, huấn luyện WGAN-GP, kiểm thử YOLO và chỉnh sửa các script như quản lý PCB, chụp ảnh, nhận dạng và phân loại.

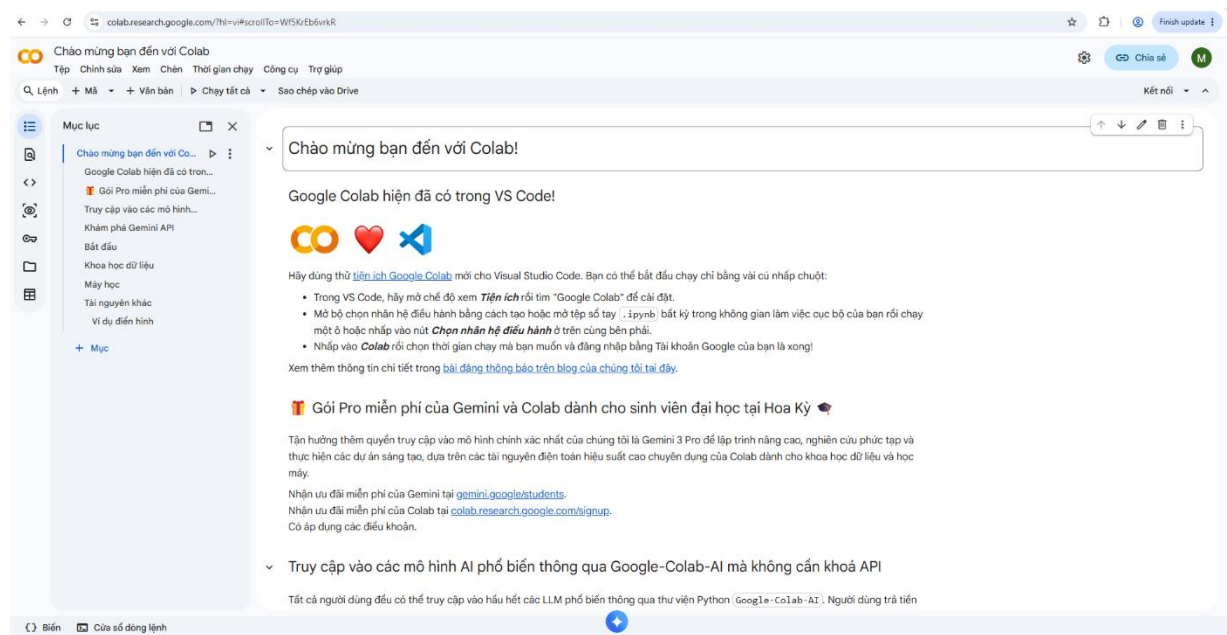


Hình 2.16. Giao diện Visual Studio Code dùng trong quá trình lập trình

2.8. NỀN TẢNG GOOGLE COLAB

Google Colab được dùng để chạy các notebook huấn luyện mô hình nhờ có sẵn môi trường Python và khả năng dùng GPU.

Trong đồ án, Colab phục vụ huấn luyện WGAN-GP và YOLO. Sau khi huấn luyện, mô hình tốt nhất được lưu dưới dạng trọng số để kiểm thử và tích hợp vào Unity.

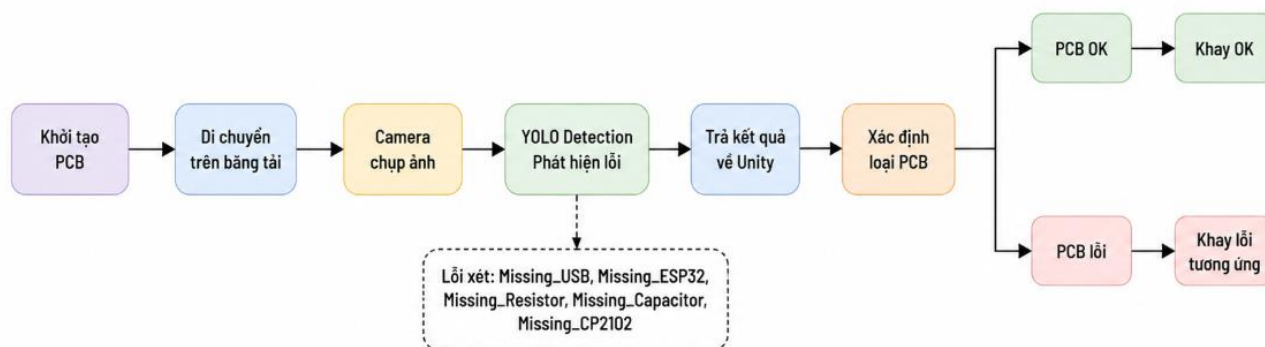


Hình 2.17. Môi trường Google Colab dùng để huấn luyện mô hình

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1. THIẾT KẾ TỔNG THỂ HỆ THỐNG

3.1.1. Tổng quan quá trình xây dựng hệ thống



Hình 3.1. Nguyên lý hoạt động của hệ thống

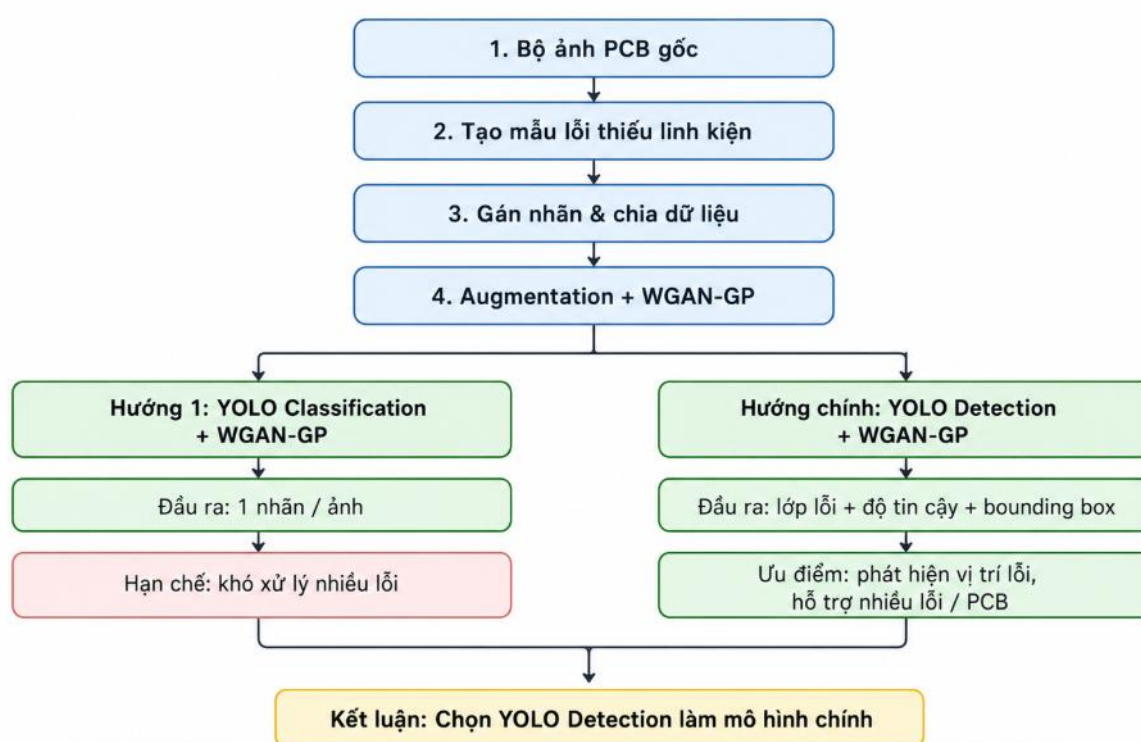
• Nguyên lý hoạt động:

- ❖ Hệ thống được xây dựng để mô phỏng logic kiểm tra và phân loại PCB tự động. Các đối tượng trong Unity như băng tải, camera, cảm biến, Cylinder và khay chứa có vai trò biểu diễn quy trình vận hành; chuyển động và phân loại được điều khiển bằng script C#.
- ❖ PCB được khởi tạo tại đầu băng tải mô phỏng và di chuyển đến vùng kiểm tra theo tốc độ được lập trình.
- ❖ Khi PCB đến điểm kiểm tra, camera trong Unity chụp ảnh bề mặt PCB để làm dữ liệu đầu vào cho mô hình nhận dạng.
- ❖ Ảnh chụp được đưa vào mô hình YOLO Detection để phát hiện lỗi thiếu linh kiện. Đầu ra gồm tên lớp lỗi, độ tin cậy và tọa độ bounding box nếu có lỗi.
- ❖ Trong phạm vi đồ án, các lỗi được xét gồm Missing_USB, Missing_ESP32, Missing_Resistor, Missing_Capacitor và Missing_CP2102. PCB_OK là trường hợp không phát hiện lỗi vượt ngưỡng tin cậy.
- ❖ Kết quả nhận dạng được gửi lại Unity để hiển thị trên giao diện và lưu vào thông tin của PCB.
- ❖ Từ kết quả đã lưu, hệ thống xác định PCB thuộc nhóm đạt yêu cầu hoặc thuộc một nhóm lỗi cụ thể.
- ❖ PCB đạt yêu cầu được đưa vào khay OK. PCB lỗi được đưa vào khay tương ứng với lớp lỗi được nhận dạng.
- ❖ Khối phân loại trong Unity chạy theo logic script, chưa mô phỏng lực đẩy hoặc va chạm vật lý của xi lanh thật.

- ❖ Cách xây dựng này giúp kiểm tra luồng AI - điều khiển - phân loại, đồng thời giữ mô phỏng ổn định trong quá trình thử nghiệm.

3.2. XÂY DỰNG BỘ DỮ LIỆU VÀ CÁC PHƯƠNG PHÁP MỞ RỘNG DỮ LIỆU

- ❖ Dữ liệu ảnh ảnh hưởng trực tiếp đến khả năng học của mô hình. Nếu dữ liệu ít hoặc thiếu đa dạng, mô hình dễ nhận dạng sai khi gặp ảnh mới.
- ❖ Trong đồ án, dữ liệu được xây dựng từ ảnh bo mạch ESP32 DevKit. Do ảnh lỗi thực tế còn hạn chế, các mẫu thiếu linh kiện được tạo và sắp xếp theo từng lớp để phục vụ hai hướng huấn luyện.
- ❖ Quy trình xây dựng dữ liệu và lựa chọn hướng mô hình được thể hiện ở Hình 3.2.



Hình 3.2. Quy trình xây dựng dữ liệu và lựa chọn mô hình YOLO Detection

Đồ án khảo sát hai hướng. Hướng thứ nhất là YOLO Classification kết hợp WGAN-GP, dùng để phân loại trạng thái toàn ảnh PCB. Hướng thứ hai là YOLO Detection kết hợp WGAN-GP, dùng để phát hiện vị trí linh kiện thiếu. Sau khi so sánh, Detection được chọn làm hướng chính vì phù hợp hơn với yêu cầu hiển thị lỗi và phân loại trong Unity.

• Thu thập dữ liệu và xây dựng các lớp lỗi PCB

Bộ dữ liệu ban đầu được xây dựng từ ảnh PCB ESP32 DevKit ở góc nhìn từ trên xuống. Góc nhìn này phù hợp vì các linh kiện cần kiểm tra nằm rõ trên bề mặt bo mạch.

Một số ảnh PCB được tạo hoặc chỉnh sửa để thống nhất bố cục, kích thước và vùng nhìn. Trước khi đưa vào huấn luyện, ảnh được kiểm tra thủ công để loại bỏ mẫu mờ, méo hoặc sai cấu trúc bo mạch.

Từ ảnh PCB ban đầu, các trường hợp thiếu linh kiện được tạo bằng cách chỉnh sửa ảnh trong Photopea. Các mẫu lỗi được tạo sao cho vị trí thiếu linh kiện đủ rõ để phục vụ huấn luyện mô hình.

Các ảnh sau khi chỉnh sửa được chia theo từng trạng thái lỗi. Với hướng Classification, ảnh được đặt trong thư mục tương ứng với tên lớp. Với hướng Detection, ảnh được gán bounding box tại vị trí linh kiện bị thiếu.

3.3. HUẤN LUYỆN MÔ HÌNH YOLO CLASSIFICATION KẾT HỢP WGAN-GP

3.3.1. Chuẩn bị bộ dữ liệu cho YOLO Classification

YOLO Classification dùng dữ liệu theo cấu trúc thư mục, mỗi thư mục tương ứng một lớp. Hướng này không cần gán bounding box nên bước chuẩn bị dữ liệu đơn giản hơn Detection.

Bộ dữ liệu gồm các lớp PCB_OK, PCB_Missing_USB, PCB_Missing_ESP32, PCB_Missing_Resistor, PCB_Missing_Capacitor và PCB_Missing_CP2102.

```
!find /content/dataset_640 -maxdepth 2 -type d
... /content/dataset_640
    /content/dataset_640/train
    /content/dataset_640/train/PCB_Missing_Resistor
    /content/dataset_640/train/PCB_Missing_USB
    /content/dataset_640/train/PCB_OK
    /content/dataset_640/train/PCB_Missing_Capacitor
    /content/dataset_640/train/PCB_Missing_CP2102
    /content/dataset_640/train/PCB_Missing_ESP32
    /content/dataset_640/val
    /content/dataset_640/val/PCB_Missing_Resistor
    /content/dataset_640/val/PCB_Missing_USB
    /content/dataset_640/val/PCB_OK
    /content/dataset_640/val/PCB_Missing_Capacitor
    /content/dataset_640/val/PCB_Missing_CP2102
    /content/dataset_640/val/PCB_Missing_ESP32
```

Hình 3.3. Cấu trúc thư mục dữ liệu cho YOLO Classification

Tập Train và Validation được giữ cố định giữa các lần huấn luyện để kết quả so sánh có cùng điều kiện dữ liệu.

- **Cấu hình huấn luyện cơ bản:**

Mô hình: YOLO11n-cls.

Số epoch: 100.

Kích thước ảnh: 640×640 pixel.

```

model = YOLO("yolo11n-cls.pt")

model.train(
    data=str(DATA_DIR),
    epochs=100,
    imgsz=640,
    batch=16,
    patience=50,
    project=SAVE_DIR,
    name="train",
    plots=True,
    exist_ok=True
)

RESULT_DIR = Path(SAVE_DIR) / "train"

print("Train xong")
print("Best:", RESULT_DIR / "weights" / "best.pt")
print("Last:", RESULT_DIR / "weights" / "last.pt")

Ultralytics 8.4.87 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugument, batch=16,
train: /content/Dataset_Classification/Dataset_Classification/train... found 1520 images in 6 classes
val: /content/Dataset_Classification/Dataset_Classification/val... found 382 images in 6 classes
test: None...
Overriding model.yaml nc=80 with nc=6

      from  n  params  module  arguments
0         -1  1     464  ultralytics.nn.modules.conv.Conv  [3, 16, 3, 2]
1         -1  1     4672 ultralytics.nn.modules.conv.Conv  [16, 32, 3, 2]
2         -1  1     6640 ultralytics.nn.modules.block.C3k2  [32, 64, 1, False, 0.25]
3         -1  1    36992 ultralytics.nn.modules.conv.Conv  [64, 64, 3, 2]

```

Hình 3.4. Cấu hình huấn luyện YOLO Classification trên Google Colab

Sau huấn luyện, mô hình tạo ra hai tệp trọng số best.pt và last.pt trong thư mục content/drive/MyDrive/YOLO_Classification_Result. Các tệp này được dùng để đánh giá hướng Classification và so sánh với hướng Detection.

3.3.2. Huấn luyện mô hình WGAN-GP

Trong đồ án, WGAN-GP không dùng để phân loại PCB. Mô hình này chỉ được huấn luyện để khảo sát khả năng sinh thêm ảnh cho các lớp lỗi còn ít mẫu. Do PCB có cấu trúc linh kiện cố định, ảnh sinh chỉ được xem là dữ liệu hỗ trợ; trước khi đưa vào YOLO, các mẫu mờ, méo hoặc sai bố cục phải được loại bỏ.

Theo chương trình huấn luyện, dữ liệu đầu vào được lấy từ file ZIP trên Google Drive. Mỗi lần huấn luyện sử dụng một file ZIP tương ứng với một lớp PCB. Ảnh được giải nén, đọc bằng PIL, chuyển sang RGB và resize về 512×512 pixel trước khi đưa vào DataLoader.

- Tiền xử lý dữ liệu

Trong code, tiền xử lý gồm đọc ảnh từ thư mục đã giải nén, resize về 512×512 , chuyển sang Tensor và chuẩn hóa pixel về khoảng $[-1; 1]$. Cách chuẩn hóa này phù hợp với hàm Tanh ở đầu ra Generator, giúp ảnh thật và ảnh sinh nằm cùng miền giá trị khi đưa vào Critic.

Trong DataLoader, batch size được đặt bằng 4 do ảnh có kích thước 512×512 và bộ nhớ GPU trên Google Colab có giới hạn. Tham số drop_last=True bỏ batch cuối nếu không đủ số ảnh, nhờ đó kích thước batch ổn định khi tính Gradient Penalty.

Cấu hình huấn luyện

Thông số	Giá trị	Ý nghĩa
IMG_SIZE	512	Kích thước ảnh đầu vào và ảnh sinh ra
Z_DIM	100	Kích thước vector nhiễu đầu vào của Generator

BATCH_SIZE	4	Số ảnh trong một batch; chọn nhỏ để tránh thiếu bộ nhớ GPU
EPOCHS	150	Số vòng huấn luyện
LR	1e-4	Tốc độ học của Generator và Critic
CRITIC_ITER	5	Số lần cập nhật Critic trước mỗi lần cập nhật Generator
LAMBDA_GP	10	Hệ số của thành phần Gradient Penalty
SAVE_EVERY	5	Chu kỳ lưu trọng số và ảnh mẫu theo epoch

Các tham số chính khi huấn luyện WGAN-GP

Các tham số được chọn theo hướng ưu tiên ổn định. Kích thước 512×512 giữ lại nhiều chi tiết hơn so với 256×256 nhưng làm tăng chi phí tính toán, vì vậy batch size giảm xuống 4. Critic được cập nhật 5 lần cho mỗi lần cập nhật Generator để có đủ khả năng đánh giá khác biệt giữa ảnh thật và ảnh sinh.

- Kiến trúc Generator và Critic

Generator nhận vector nhiễu dạng $[B, 100, 1, 1]$. Mạng dùng các lớp ConvTranspose2d kết hợp BatchNorm2d và ReLU để tăng dần kích thước đặc trưng theo chuỗi $1 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$. Lớp cuối dùng Tanh để sinh ảnh RGB kích thước 512×512 .

Critic nhận ảnh RGB 512×512 và dùng các lớp Conv2d để giảm dần kích thước đặc trưng. Các lớp trung gian dùng InstanceNorm2d và LeakyReLU. Khác với Discriminator trong GAN truyền thống, Critic không dùng Sigmoid và không trả về xác suất thật - giả; đầu ra là một điểm số để ước lượng khoảng cách Wasserstein giữa ảnh thật và ảnh sinh.

- Hàm mất mát và Gradient Penalty

Ở mỗi lần cập nhật Critic, code tính điểm trung bình của ảnh thật và ảnh sinh, sau đó tối ưu loss theo công thức WGAN-GP. Gradient Penalty được tính trên ảnh nội suy giữa ảnh thật và ảnh sinh để ràng buộc chuẩn gradient gần 1. Nhờ đó, quá trình huấn luyện ổn định hơn so với GAN cơ bản.

Đoạn mã chính thể hiện cách tính loss của WGAN-GP trong chương trình:

```
critic_real = D(real).mean()
critic_fake = D(fake.detach()).mean()
gp = gradient_penalty(D, real, fake.detach())
loss_d = -(critic_real - critic_fake) + LAMBDA_GP * gp
loss_g = -D(fake).mean()
```

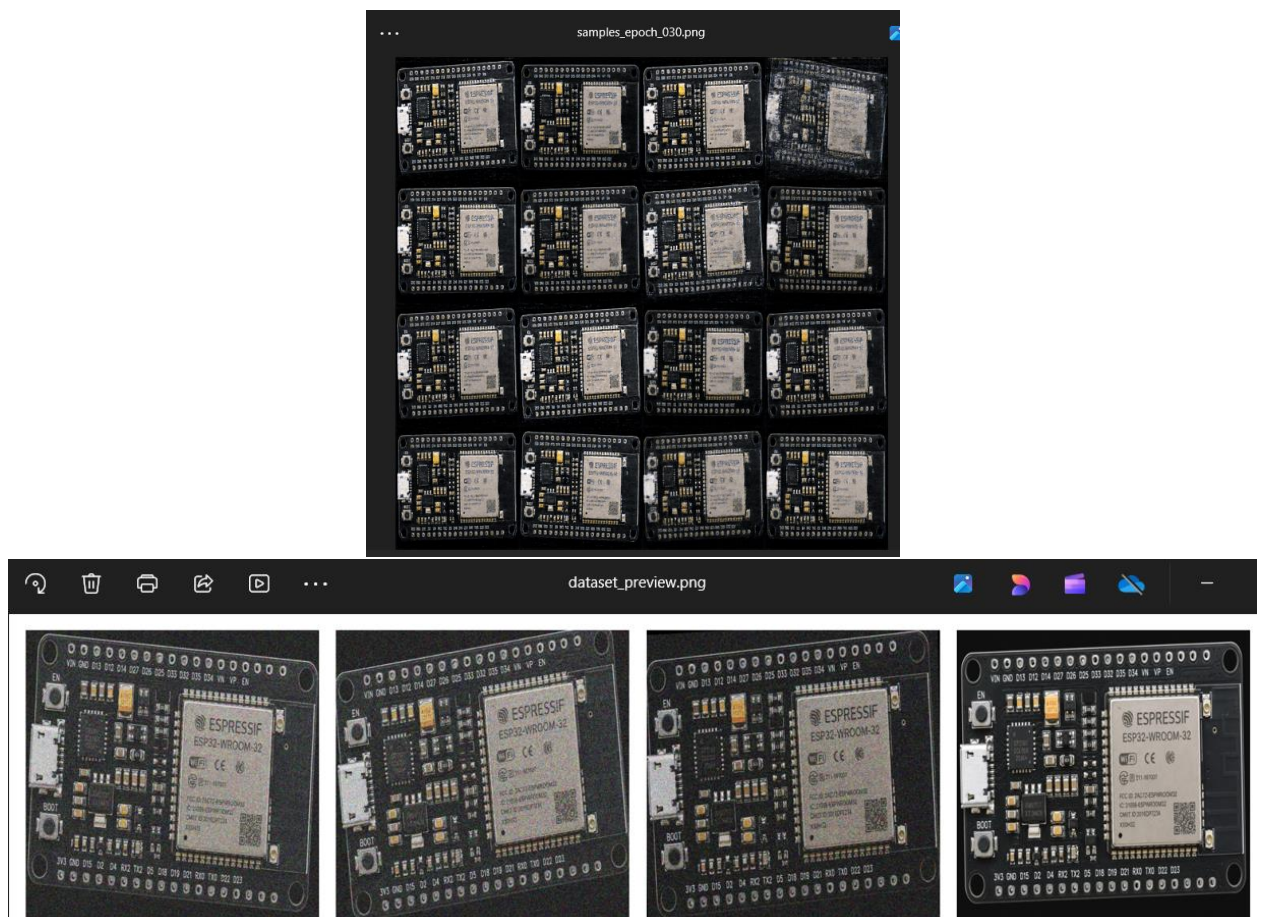

Trong đoạn code này, `loss_d` dùng để cập nhật Critic, còn `loss_g` dùng để cập nhật Generator. Dấu âm trong `loss_g` thể hiện mục tiêu của Generator là tạo ảnh có điểm Critic cao hơn, tức là ảnh sinh gần phân bố ảnh thật hơn theo đánh giá của Critic.

Quy trình huấn luyện và lưu kết quả

Quá trình huấn luyện được chạy theo từng epoch trên Google Colab. Với mỗi batch ảnh thật, chương trình cập nhật Critic 5 lần rồi cập nhật Generator 1 lần. Sau mỗi epoch, các giá trị `critic_loss`, `generator_loss`, `gradient_penalty`, `critic_real`, `critic_fake` và Wasserstein distance được ghi vào `training_log.csv` để theo dõi và vẽ biểu đồ.

Chương trình lưu `last_G.pth` và `last_D.pth` sau mỗi epoch, đồng thời lưu `last_checkpoint.pth` để có thể huấn luyện tiếp nếu Colab bị ngắt. Cứ sau 5 epoch, trọng số Generator và Critic được lưu lại dưới dạng `G_epoch_xxx.pth` và `D_epoch_xxx.pth`, kèm ảnh mẫu sinh từ `fixed_noise` để quan sát sự thay đổi chất lượng ảnh qua từng giai đoạn.

Trong code, `best_G.pth` được chọn theo điểm `critic_fake` trung bình cao nhất. Tuy nhiên, với bài toán PCB, điểm Critic chỉ nên xem là thông tin tham khảo. Khi chọn epoch để sinh ảnh bổ sung dữ liệu, cần kết hợp điểm số, biểu đồ loss và quan sát ảnh. Ảnh bị méo hình PCB, sai vị trí linh kiện hoặc nhiễu nhiều không nên đưa vào tập huấn luyện YOLO.

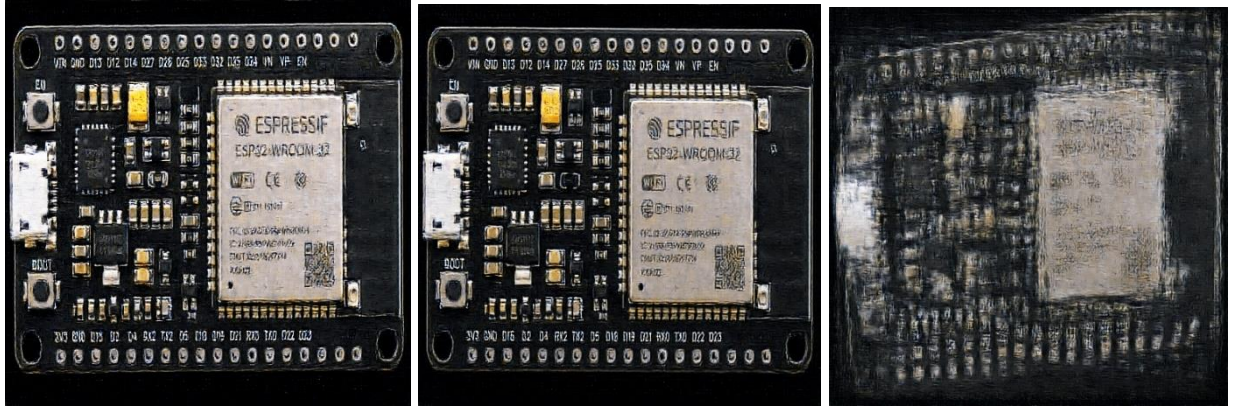


Hình 3.5. Kết quả theo dõi quá trình huấn luyện WGAN-GP trên Google Colab

- `dataset_preview.png` và `samples_epoch_30`

3.3.3. Sinh ảnh và lọc dữ liệu sinh bởi WGAN-GP

Sau huấn luyện, Generator được nạp lại từ một epoch được chọn, ví dụ G_epoch_110.pth trong chương trình. Mỗi ảnh mới được sinh từ một vector nhiễu ngẫu nhiên 100 chiều. Critic đồng thời chấm điểm ảnh sinh và điểm này được ghi trong tên file để thuận tiện cho việc sắp xếp, kiểm tra và chọn lọc.



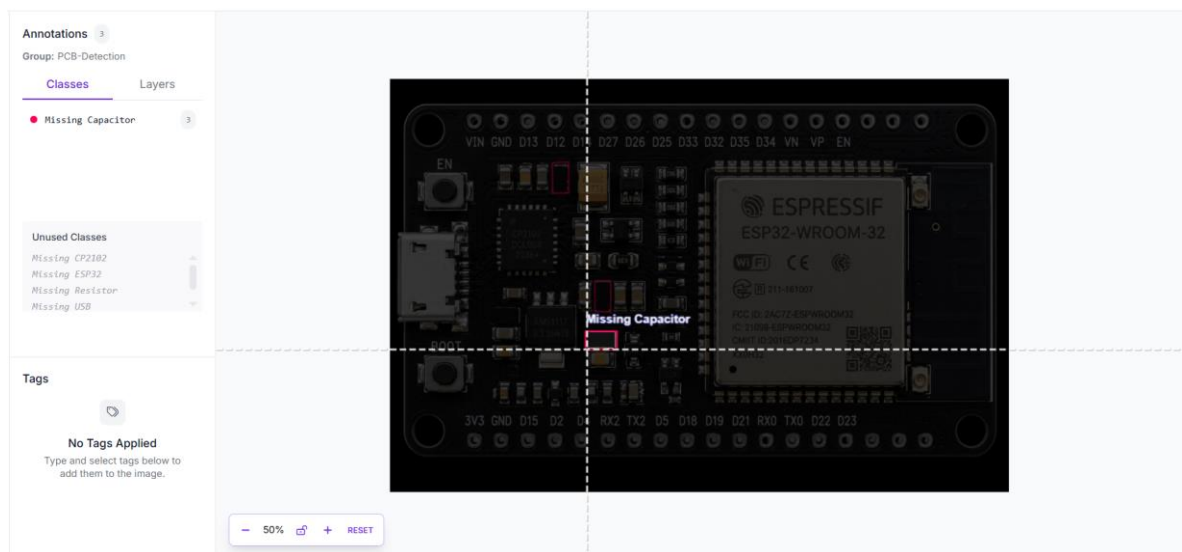
Hình 3.6. Một số ảnh PCB sinh bởi WGAN-GP sau khi huấn luyện

Kết quả cho thấy WGAN-GP học được hình dạng tổng thể của PCB, nhưng nhiều ảnh sinh vẫn bị mờ, méo hoặc sai chi tiết do dữ liệu đầu vào còn ít. Vì vậy, ảnh sinh không được dùng hàng loạt mà phải lọc trước khi bổ sung vào dataset. Trong báo cáo, WGAN-GP được xem là hướng khảo sát hỗ trợ dữ liệu; mô hình chính để kiểm tra PCB vẫn là YOLO Detection.

3.4. HUẤN LUYỆN MÔ HÌNH YOLO DETECTION KẾT HỢP WGAN-GP

3.4.1. Chuẩn bị dữ liệu cho YOLO Detection

Khác với Classification, YOLO Detection cần nhãn vị trí cho đối tượng cần phát hiện. Với mỗi ảnh lỗi, vị trí linh kiện bị thiếu được gán bằng bounding box kèm tên lớp tương ứng.

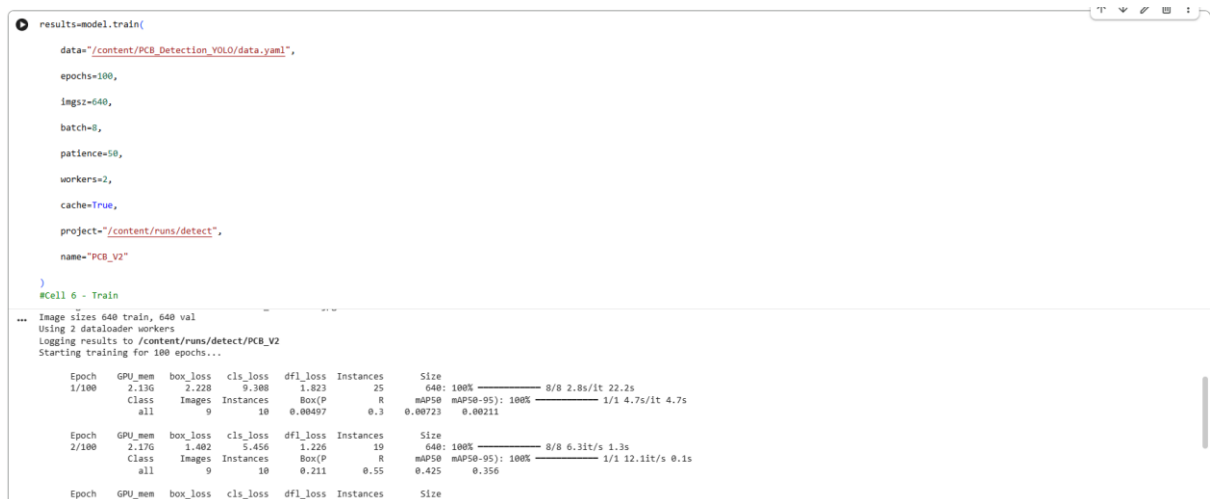


Hình 3.7. Gán nhãn bounding box cho dữ liệu YOLO Detection

Các lớp Detection gồm Missing_USB, Missing_ESP32, Missing_Resistor, Missing_Capacitor và Missing_CP2102. Với PCB_OK, mô hình không cần phát hiện đối tượng lỗi vượt ngưỡng tin cậy. Trong đồ án, các mẫu lỗi đơn được dùng để kiểm chứng nguyên lý; trường hợp nhiều lỗi được đưa vào hướng phát triển.

3.4.2. Huấn luyện YOLO Detection trên tập dữ liệu gốc

Sau khi gán nhãn, dữ liệu được chia thành Train, Validation và Test. Mô hình YOLO11 Detection được huấn luyện với kích thước ảnh 640×640 pixel. Các chỉ số Precision, Recall, mAP50 và mAP50-95 được dùng để đánh giá khả năng phát hiện lỗi.



Hình 3.8. Quá trình huấn luyện YOLO Detection trên Google Colab

3.4.3. Kết hợp WGAN-GP để mở rộng dữ liệu Detection

Ảnh sinh bởi WGAN-GP có thể làm dữ liệu đa dạng hơn. Tuy nhiên, với Detection, ảnh sinh phải giữ đúng bố cục để bounding box còn chính xác. Ảnh bị lệch, méo hoặc sai vị trí linh kiện sẽ bị loại. Với ảnh đạt yêu cầu, nhãn có thể được kế thừa hoặc hiệu chỉnh lại trước khi huấn luyện.

3.4.4. Xuất mô hình và chuẩn bị tích hợp Unity

Sau huấn luyện, mô hình Detection tốt nhất được lưu dưới dạng best.pt và có thể chuyển sang ONNX để tích hợp vào Unity. Đầu ra gồm tên lỗi, độ tin cậy và tọa độ bounding box. Ở hướng phát triển, hệ thống có thể ưu tiên lỗi có confidence cao, đưa PCB đi sửa rồi kiểm tra lại để phát hiện các lỗi còn lại.



Hình 3.9. Kết quả YOLO Detection trên ảnh kiểm thử PCB

3.5. SO SÁNH HAI HƯỚNG VÀ LỰA CHỌN MÔ HÌNH CHÍNH

3.5.1. Tiêu chí so sánh

Hai hướng được so sánh theo các tiêu chí: độ chính xác, khả năng xác định vị trí lỗi, khả năng mở rộng sang PCB có nhiều lỗi, độ phức tạp khi chuẩn bị dữ liệu và mức độ phù hợp khi tích hợp vào Unity.

YOLO Classification + WGAN-GP có ưu điểm là dữ liệu dễ chuẩn bị, nhưng đầu ra chỉ là nhãn toàn ảnh. Khi một PCB có nhiều lỗi thiếu linh kiện, hướng này phải tạo thêm các lớp tổ hợp, khiến số lớp tăng nhanh và khó thu thập đủ mẫu cho từng tổ hợp.

YOLO Detection + WGAN-GP yêu cầu gán nhãn công phu hơn, nhưng đầu ra chi tiết hơn: mỗi lỗi có class, confidence và bounding box. Hướng này phù hợp hơn với kiểm tra PCB vì cho phép khoanh vùng lỗi và có thể mở rộng sang trường hợp nhiều lỗi trên một ảnh.

3.5.2. Lý do lựa chọn YOLO Detection kết hợp WGAN-GP

YOLO Detection được chọn làm mô hình chính vì đầu ra của nó phù hợp trực tiếp với yêu cầu vận hành. Mô hình không chỉ cho biết PCB lỗi loại gì mà còn chỉ ra vị trí lỗi trên ảnh. Thông tin này được dùng để hiển thị bounding box trong Unity và làm cơ sở cho logic phân loại.

Ở hướng phát triển, hệ thống có thể làm việc theo quy trình kiểm tra lặp: phát hiện lỗi có độ tin cậy cao, đưa PCB đến công đoạn sửa, rồi kiểm tra lại. Nếu còn lỗi khác, hệ thống tiếp tục phát hiện và xử lý; nếu không còn lỗi, PCB được đưa vào nhóm OK. Trong đồ án hiện tại, các mẫu lỗi đơn chỉ dùng để chứng minh nguyên lý, chưa triển khai quy trình sửa lỗi lặp ngoài thực tế.

Điểm này làm cho Detection phù hợp hơn Classification. Với Classification, một PCB thiếu nhiều linh kiện sẽ khó gán vào một lớp duy nhất. Với Detection, mỗi linh kiện thiếu được xem là một đối tượng riêng, nên mô hình có thể trả về danh sách lỗi thay vì ép toàn ảnh vào một nhãn tổng thể.

3.6. XÂY DỰNG HỆ THỐNG MÔ PHỎNG TRÊN UNITY 3D

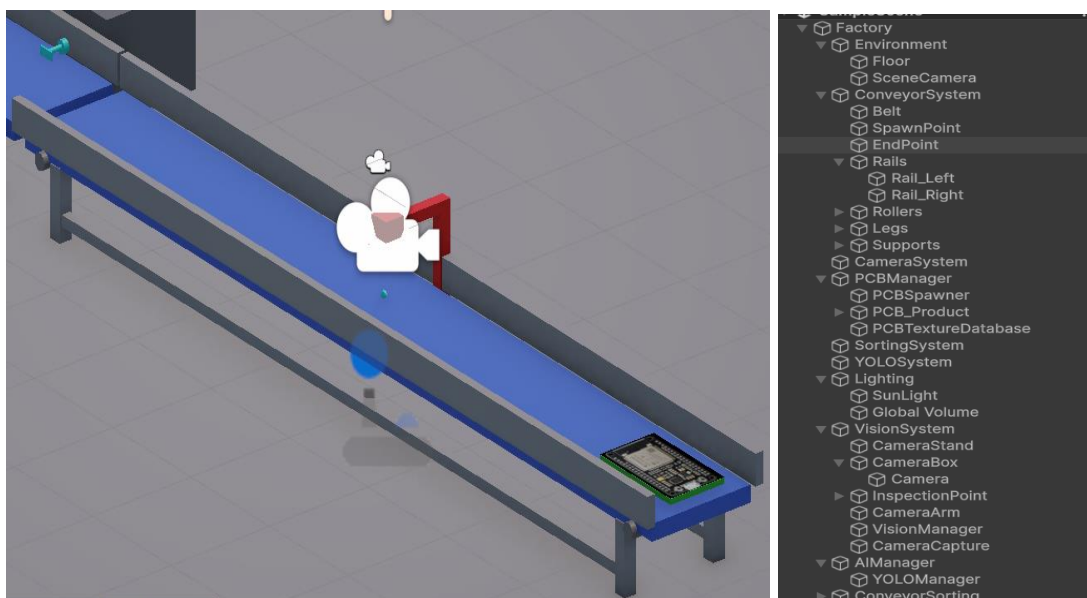
Sau khi huấn luyện mô hình YOLO Detection, hệ thống được tích hợp vào Unity 3D để mô phỏng quá trình kiểm tra và phân loại PCB. Các đối tượng như băng tải, camera, cảm biến, Cylinder và khay chứa được xây dựng để biểu diễn dây chuyền ở mức trực quan.

Cần lưu ý rằng mô phỏng trong đồ án tập trung vào logic vận hành. Chuyển động của PCB và thao tác phân loại đều do script C# điều khiển, chưa mô phỏng chi tiết lực, ma sát, va chạm hoặc động học cơ cấu.

3.6.1. Thiết kế môi trường mô phỏng

Môi trường Unity được xây dựng để thể hiện các bước làm việc của hệ thống. Các đối tượng được đặt tên theo chức năng và liên kết với script điều khiển tương ứng. Hệ thống chia thành hai khối chính: khối kiểm tra PCB và khối phân loại sản phẩm.

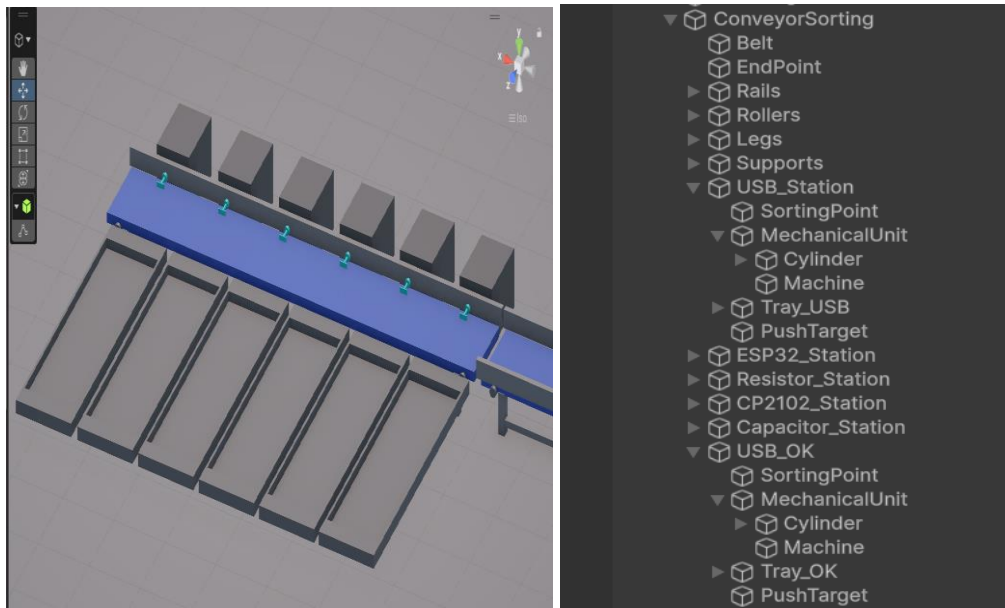
❖ Khối kiểm tra PCB



Hình 3.10. Khối kiểm tra PCB và các đối tượng trong Unity Scene

- **Khối kiểm tra PCB được bố trí tại đầu băng tải, gồm các thành phần:**
 - + Cụm băng tải mô phỏng: chân đế, con lăn, thanh chắn và mặt băng tải.
 - + Cụm camera chụp ảnh PCB.
 - + Sản phẩm PCB cần kiểm tra.
 - + Sensor vị trí/inspectionPoint dùng làm mốc kích hoạt chụp ảnh.
 - + Mô hình AI nhận dạng lỗi thiếu linh kiện.
- ➔ Khối kiểm tra thực hiện các chức năng: đưa PCB đến vị trí kiểm tra, chụp ảnh, gọi mô hình YOLO Detection, hiển thị kết quả và lưu trạng thái PCB.

❖ Khối phân loại sản phẩm



Hình 3.11. Khối phân loại sản phẩm và các đối tượng trong Unity Scene

- **Khối phân loại được bố trí ở cuối băng tải, gồm:**

- + Cụm băng tải mô phỏng ở khu vực phân loại.
- + Đối tượng Cylinder biểu diễn cơ cấu phân loại.
- + Khối Sorting gồm các khay chứa PCB sau khi phân loại.
 - ✓ Các khay tương ứng với Missing_USB, Missing_ESP32, Missing_Resistor, Missing_CP2102, Missing_Capacitor và PCB_OK.
 - ✓ Mỗi khay được thiết lập sức chứa 6 PCB trong mô phỏng.

➔ Sau khi đọc kết quả từ YOLO Detection, script phân loại đưa PCB vào khay tương ứng với trạng thái đã lưu.

3.6.2. Xây dựng logic điều khiển hệ thống

Các chức năng điều khiển được viết bằng C#. Hệ thống gồm ba nhóm chính: quản lý PCB, kiểm tra PCB và phân loại sản phẩm. Mỗi nhóm có nhiệm vụ riêng, nhưng dữ liệu được truyền liên tục giữa các nhóm.

Khối chức năng	Scripts chính	Chức năng
Quản lý PCB	PCBSpawner ; PCBMove ; PCBInfo	Khởi tạo PCB và điều khiển quá trình di chuyển
Kiểm tra PCB	VisionManager ; CameraCapture ; PCBYOLOManager	Thu nhận ảnh và thực hiện nhận dạng bằng YOLO Detection
Phân loại sản phẩm	SortingStationController ; TrayController	Điều khiển cơ cấu phân loại và quản lý khay chứa

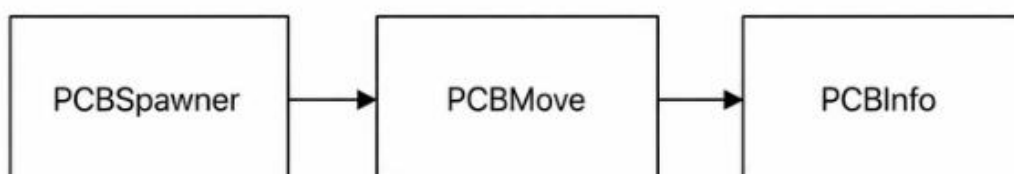
➔ Ngoài các script chính, hệ thống còn dùng các script hỗ trợ như PCBInfo, PCBTextureChanger, PCBTextureDatabase, BoundingBoxRenderer và

CameraPreviewUI để lưu thông tin PCB, thay đổi ảnh bề mặt, vẽ bounding box và hiển thị kết quả.

- **Khối quản lý PCB :**

+ Khối quản lý PCB gồm PCBSpawner và PCBMove. PCBSpawner tạo PCB tại đầu băng tải theo thời gian thiết lập, còn PCBMove điều khiển quá trình di chuyển của PCB bằng script.

+ PCBInfo lưu trạng thái thật và trạng thái dự đoán của từng PCB. PCBTextureChanger và PCBTextureDatabase dùng để thay đổi ảnh bề mặt PCB trước khi kiểm tra.

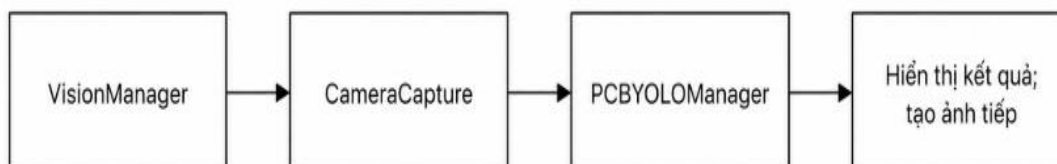


Hình 3.12. Logic hoạt động của khối quản lý PCB

- **Khối kiểm tra PCB :**

Khối kiểm tra gồm VisionManager, CameraCapture và PCBYOLOManager. VisionManager điều phối thời điểm kiểm tra, CameraCapture chụp ảnh PCB, còn PCBYOLOManager chạy mô hình YOLO Detection để nhận dạng lỗi.

BoundingBoxRenderer được dùng để vẽ khung lỗi trên ảnh kết quả. CameraPreviewUI hiển thị ảnh chụp và thông tin nhận dạng trên giao diện Unity.



Hình 3.13. Logic hoạt động của khối kiểm tra PCB

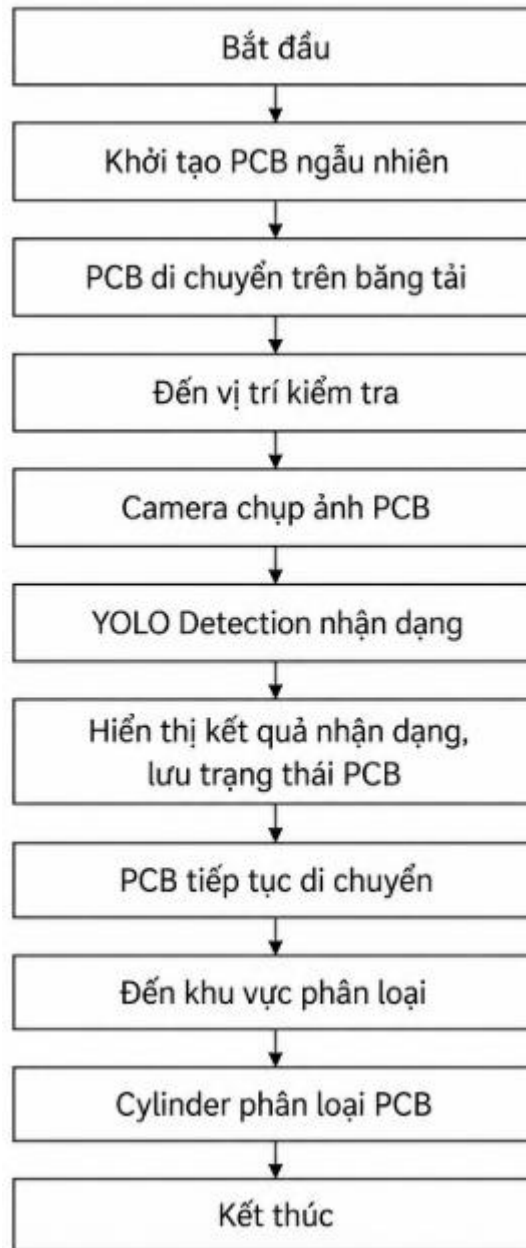
- **Khối phân loại sản phẩm :**

Khối phân loại gồm SortingStationController và TrayController. SortingStationController đọc kết quả nhận dạng đã lưu và điều khiển PCB đi vào khay tương ứng. TrayController quản lý số lượng và vị trí PCB trong từng khay.



Hình 3.14. Logic hoạt động của khối phân loại sản phẩm

3.6.3. Thiết kế quy trình kiểm tra và phân loại PCB



Hình 3.15. Quy trình hoạt động tổng thể của hệ thống mô phỏng

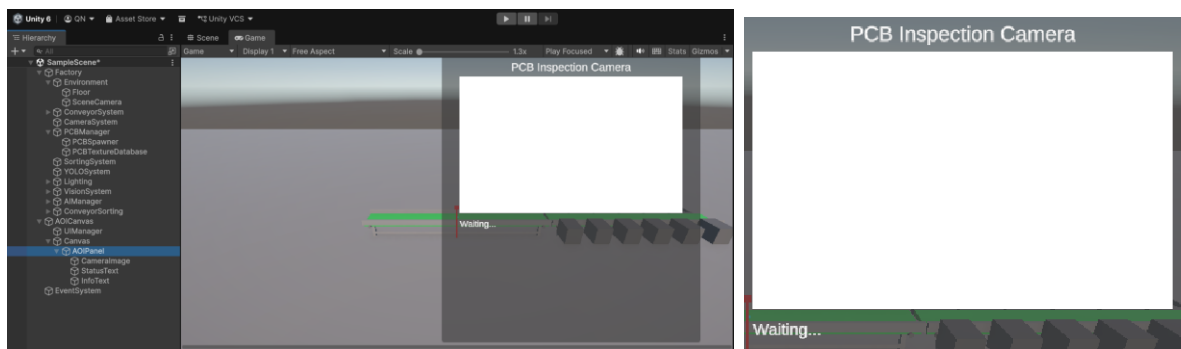
Quy trình hoạt động của hệ thống gồm các bước:

- Bước 1: PCB được khởi tạo tại đầu băng tải mô phỏng và bắt đầu di chuyển theo tốc độ do script thiết lập.
- Bước 2: Khi PCB đến vị trí kiểm tra, hệ thống giữ PCB tại điểm chụp và camera thực hiện thu ảnh.
- Bước 3: Ảnh được đưa vào YOLO Detection để phát hiện lỗi thiếu linh kiện. Kết quả gồm class, confidence và bounding box nếu có lỗi.

- Bước 4: Kết quả nhận dạng được hiển thị trên giao diện và lưu vào đối tượng PCB để phục vụ phân loại.
- Bước 5: PCB tiếp tục di chuyển đến khu vực phân loại theo logic script.
- Bước 6: Hệ thống đọc trạng thái đã lưu và đưa PCB vào khay tương ứng. Trong hướng phát triển, PCB sau khi sửa lỗi có thể quay lại vùng kiểm tra để tiếp tục phát hiện lỗi còn lại.

Quy trình này dùng để kiểm tra sự phối hợp giữa YOLO Detection và logic điều khiển trong Unity. Do hệ thống chưa mô phỏng vật lý chi tiết, kết quả vận hành được đánh giá chủ yếu qua ba điểm: nhận dạng đúng, lưu đúng trạng thái và đưa PCB vào đúng khay.

3.6.4. Thiết kế màn hình UI theo dõi và hiển thị kết quả



Hình 3.16. Màn hình chờ hiển thị khi chưa chạy mô phỏng



Hình 3.17 Màn hình theo dõi kết quả khi ô hình đang chạy

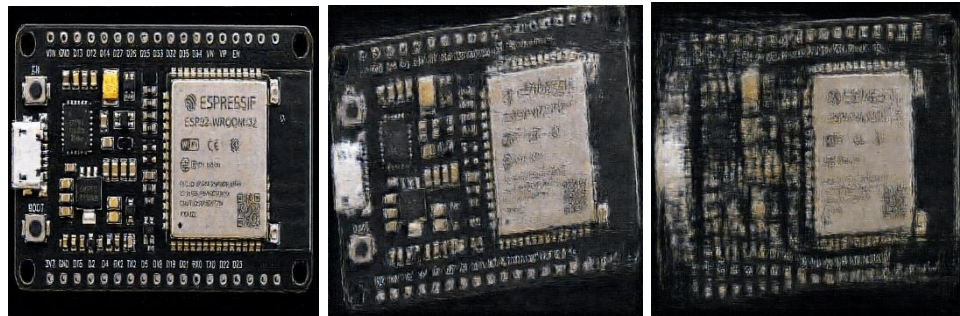
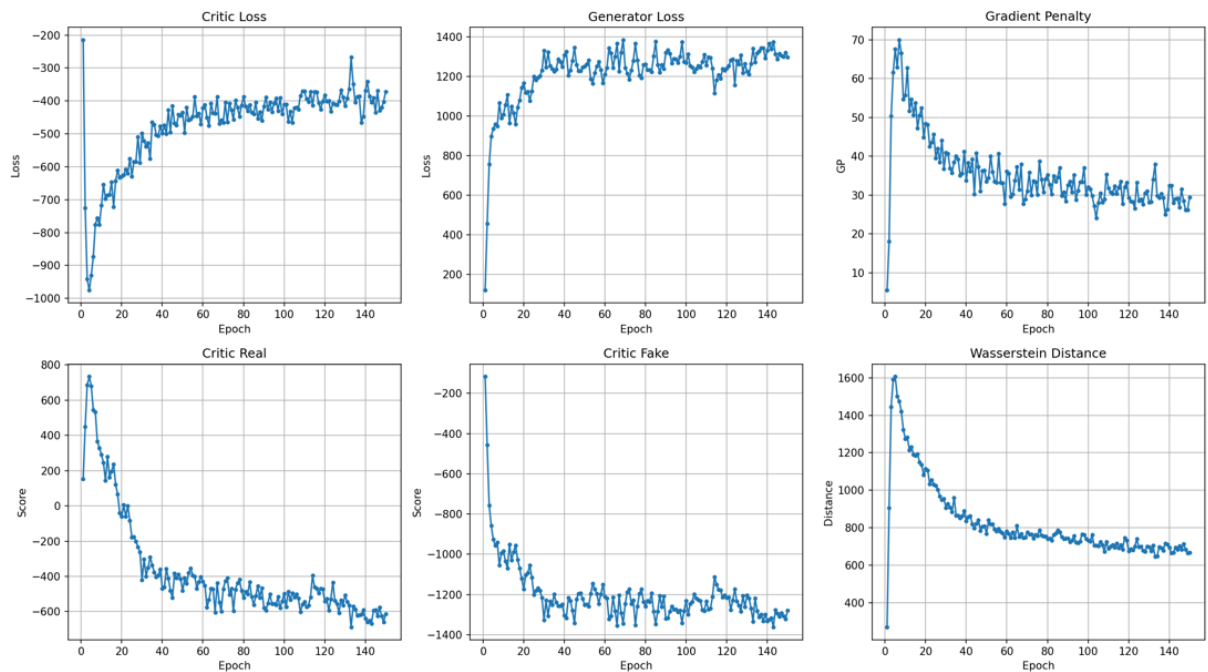
Giao diện UI được thiết kế để hiển thị kết quả kiểm tra PCB sau khi camera chụp ảnh và mô hình YOLO xử lý. Trên giao diện, hệ thống hiển thị ảnh kiểm tra, vị trí lỗi nếu có, lớp dự đoán, độ tin cậy và trạng thái sản phẩm. Nếu sản phẩm đạt yêu cầu, trạng thái hiển thị là OK; nếu phát hiện lỗi thiếu linh kiện, trạng thái hiển thị là NG. Giao diện cũng hỗ trợ đối chiếu kết quả dự đoán với nhãn thực tế để đánh giá độ chính xác của mô hình.

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. KẾT QUẢ HUẤN LUYỆN WGAN-GP

WGAN-GP được dùng để khảo sát khả năng sinh thêm ảnh PCB lỗi. Mô hình này không có chỉ số Accuracy như YOLO, vì vậy phần đánh giá dựa trên diễn biến loss, Gradient Penalty, điểm Critic của ảnh thật/ảnh sinh và quan sát trực tiếp ảnh sinh ra. Các chỉ số này chỉ cho biết xu hướng huấn luyện; chất lượng ảnh PCB vẫn phải kiểm tra bằng mắt vì chỉ cần sai cấu trúc linh kiện thì ảnh sinh không nên đưa vào tập huấn luyện.

Thông số/kết quả	Giá trị	Đánh giá
Số epoch	150	Đủ để quan sát xu hướng học của Generator và Critic.
Critic loss cuối	-371,58	Giá trị còn dao động. Đây là đặc điểm thường gặp khi huấn luyện GAN, không thể đọc như loss của YOLO.
Generator loss cuối	1298,72	Tăng và dao động ở vùng cao, cho thấy Generator vẫn còn khó đánh lừa Critic.
Gradient Penalty cuối	29,46	Giúp ổn định ràng buộc Lipschitz, nhưng giá trị vẫn dao động nên quá trình sinh ảnh chưa thật sự ổn định.
Critic real / fake cuối	-614,21 / -1280,41	Điểm ảnh thật và ảnh sinh còn cách nhau khá xa; ảnh sinh chưa đạt mức giống ảnh thật hoàn toàn.
Wasserstein distance cuối	666,20	Khoảng cách phân bố vẫn còn lớn, cho thấy WGAN-GP mới học được tổng thể hơn là chi tiết chính xác.



Hình 4.1. Kết quả huấn luyện và ảnh sinh của WGAN-GP

Nhìn chung, WGAN-GP đã học được hình dạng tổng thể của bo mạch: nền PCB, bố cục lớn và vị trí tương đối của một số vùng linh kiện. Tuy nhiên, các chi tiết nhỏ như chân linh kiện, đường biên linh kiện và vùng mạch in còn mờ, nhiều hoặc chưa ổn định.

→ Kết quả này cho thấy mô hình chưa đảm bảo được chất lượng ảnh đầu ra và cũng chưa đủ tốt để sử dụng toàn bộ ảnh sinh như ảnh thật → cần sinh nhiều ảnh, lọc thủ công các ảnh ít méo nhất, sau đó mới đưa một phần nhỏ vào tập huấn luyện YOLO

➔ Trong đồ án này WGAN-GP được xem là phương pháp hỗ trợ mở rộng dữ liệu, không phải thành phần quyết định chính của hệ thống.

4.2. KẾT QUẢ THỰC NGHIỆM YOLO CLASSIFICATION

4.2.1. YOLO Classification trên dữ liệu gốc

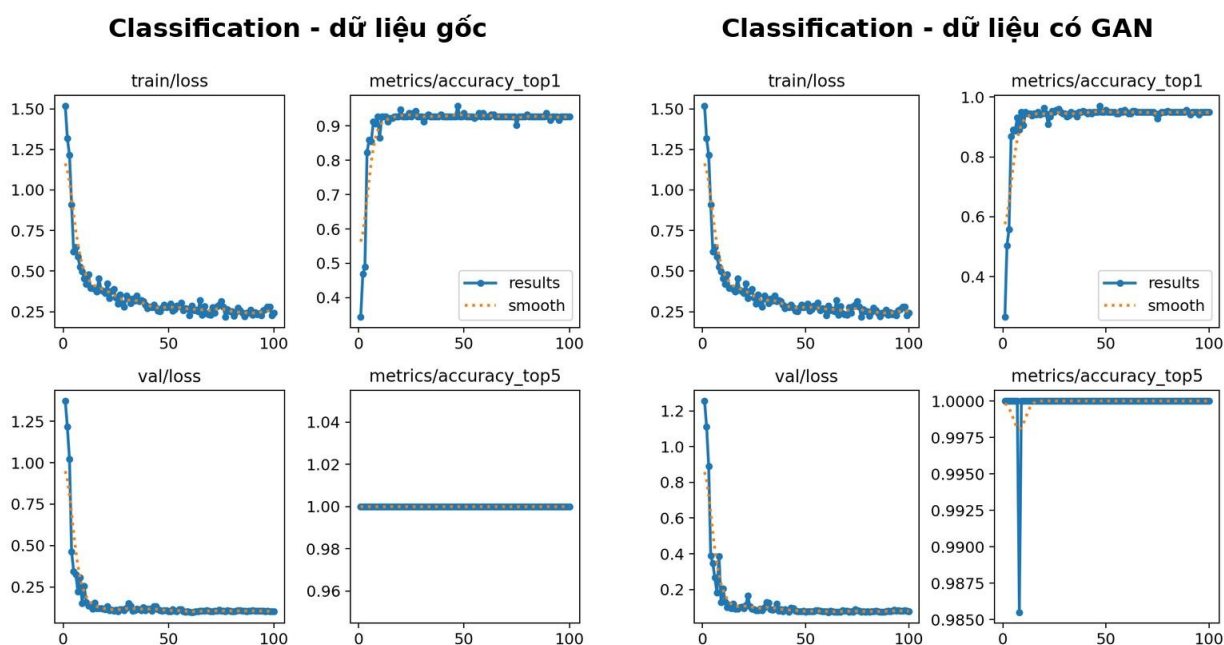
YOLO Classification được thử nghiệm để đánh giá hướng phân loại toàn ảnh. Với cách này, mỗi ảnh PCB chỉ có một nhãn duy nhất. Chỉ số quan trọng nhất là Top-1 Accuracy, vì nó cho biết lớp có xác suất cao nhất có trùng với nhãn thật hay không. Top-5 Accuracy chỉ dùng để tham khảo vì bộ dữ liệu có ít lớp; chỉ số này dễ đạt cao và không phản ánh rõ khả năng phân biệt giữa các lỗi.

Chỉ số	Kết quả	Nhận xét
Top-1 Accuracy cuối	92,71%	Khá tốt đối với bài toán lỗi đơn. Mô hình đã học được phần lớn đặc trưng giữa các lớp PCB.
Top-5 Accuracy cuối	100%	Không có nhiều ý nghĩa phân biệt vì số lớp ít. Không dùng làm căn cứ chính để chọn mô hình.
Train loss cuối	0,24185	Loss thấp, quá trình học ổn định.
Validation loss cuối	0,10354	Thấp hơn train loss, chưa thấy dấu hiệu quá khớp rõ rệt ở lần train này.
Accuracy trên tập đánh giá	96,35%	Kết quả nhận dạng toàn ảnh tốt với các mẫu lỗi đơn đã chuẩn bị.
Macro Precision / Recall / F1	92,07% / 91,67% / 91,57%	Mức tốt nhưng chưa đồng đều giữa tất cả lớp; một số lớp vẫn dễ nhầm hơn.

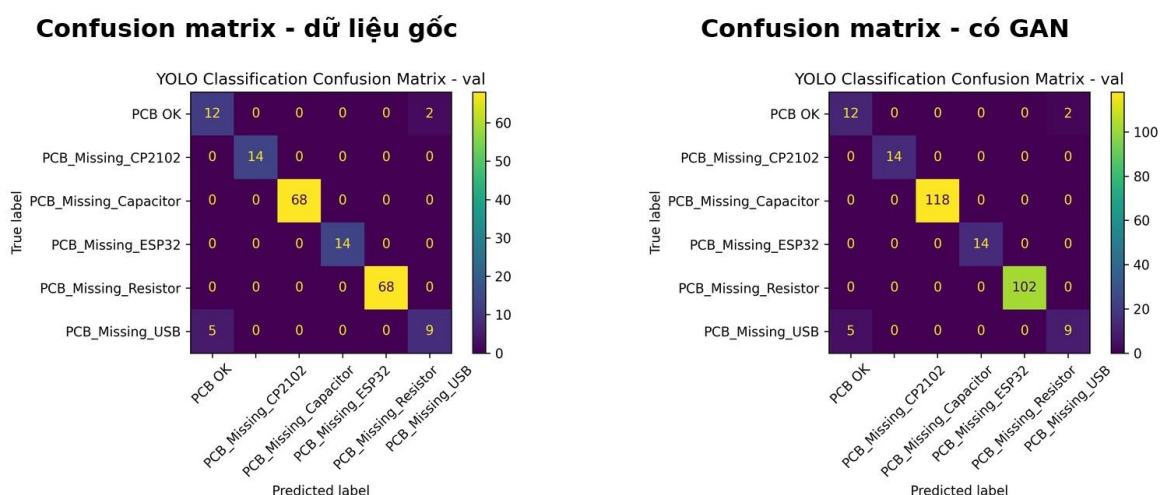
Kết quả trên cho thấy YOLO Classification có thể dùng được rất tốt nếu mỗi PCB chỉ có một trạng thái rõ ràng.

4.2.2. YOLO Classification sau khi bổ sung ảnh WGAN-GP

Chỉ số	Dữ liệu gốc	Có WGAN-GP	Chênh lệch	Đánh giá
Top-1 Accuracy cuối	92,71%	94,93%	+2,22 điểm %	Có cải thiện nhẹ.
Top-5 Accuracy cuối	100%	100%	Không đổi	Không phải chỉ số quyết định.
Validation loss cuối	0,10354	0,07630	Giảm 0,02724	Tập có WGAN-GP cho đường validation ổn hơn.
Accuracy đánh giá	96,35%	97,46%	+1,11 điểm %	Cải thiện nhưng không lớn.
Macro F1-score	91,57%	91,57%	Không đổi	Khả năng cân bằng giữa các lớp chưa cải thiện rõ.



Hình 4.2. Sơ đồ kết quả huấn luyện YOLO Classification trước và sau khi bổ sung WGAN-GP



Hình 4.3. Ma trận nhầm lẫn của YOLO Classification

Sau khi bổ sung ảnh WGAN-GP, Top-1 Accuracy và Accuracy tăng nhẹ, validation loss giảm → ảnh sinh sau khi lọc có hỗ trợ nhất định cho quá trình học. Tuy nhiên, mức cải thiện chưa đủ lớn để kết luận rằng WGAN-GP làm thay đổi rõ rệt chất lượng mô hình. Macro F1-score gần như không đổi, nghĩa là các lớp khó vẫn chưa được cải thiện đáng kể.

Hạn chế quan trọng hơn của Classification nằm ở dạng đầu ra. Mô hình chỉ trả về một nhãn cho cả ảnh, không có vị trí lỗi. Nếu một PCB thiếu nhiều linh kiện, cần tạo thêm các lớp tổ hợp như Missing_USB_CP2102 hoặc Missing_USB_Resistor_Capacitor. Số lớp sẽ tăng nhanh khi số loại lỗi tăng.

➔ Dù kết quả Classification ở tập thử nghiệm hiện tại khá tốt nhưng hướng này không phù hợp nhất với mục tiêu kiểm tra và chỉ vị trí lỗi trong hệ thống mô phỏng.

4.3. KẾT QUẢ THỰC NGHIỆM YOLO DETECTION

4.3.1. YOLO Detection trên dữ liệu gốc

YOLO Detection được đánh giá bằng Precision, Recall, mAP50, mAP50-95 và các loss trên tập train/validation. Trong bài toán kiểm tra PCB, Recall có ý nghĩa lớn vì bỏ sót lỗi là trường hợp nguy hiểm hơn báo nhầm. mAP50 cho biết khả năng phát hiện đúng ở ngưỡng IoU 0,5, còn mAP50-95 đánh giá khắt khe hơn về độ chính xác của bounding box.

Chỉ số	Kết quả	Đánh giá
Precision	98,82%	Rất tốt. Số dự đoán nhầm thấp.
Recall	100%	Rất tốt. Trên tập đánh giá, mô hình không bỏ sót lỗi được gán nhãn.
mAP50	99,50%	Rất cao, cho thấy mô hình phát hiện đúng loại lỗi và vị trí tương đối tốt.
mAP50-95	82,56%	Tốt nhưng chưa hoàn hảo; bounding box ở các ngưỡng IoU cao vẫn có sai lệch.
Train box/cls/df1 loss cuối	0,75797 / 0,36220 / 0,84971	Loss đã giảm về mức chấp nhận được.
Val box/cls/df1 loss cuối	0,97425 / 0,40740 / 0,90971	Val loss cao hơn train loss nhưng chưa cho thấy mất ổn định nghiêm trọng.

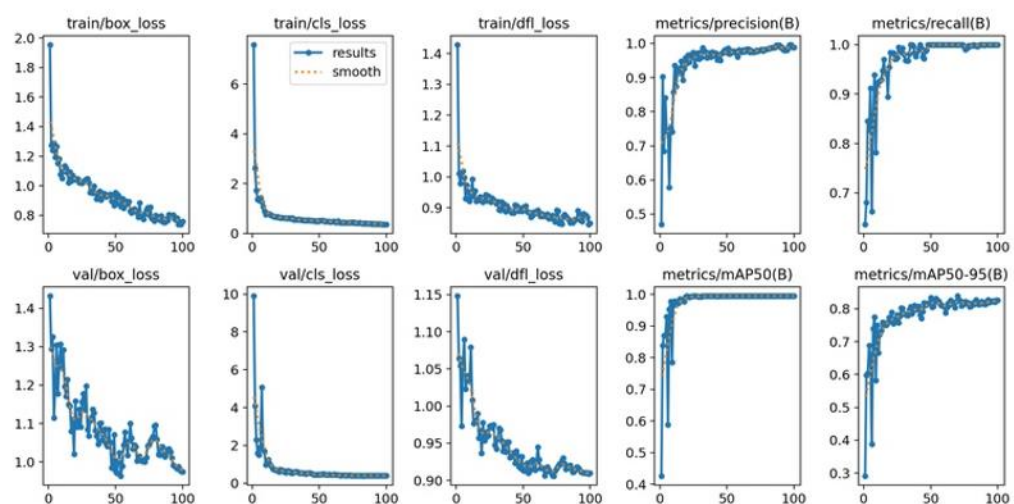
Với dữ liệu gốc, kết quả Detection đã đủ tốt cho mục tiêu mô phỏng. Precision, Recall và mAP50 đều ở mức cao. Điểm còn cần cải thiện là mAP50-95, vì chỉ số này nhạy với độ khớp bounding box.

→ Mô hình đã tìm đúng vùng lỗi, nhưng khung bao ở một số ảnh chưa thật sát với nhãn. Với hệ thống Unity hiện tại, mức này vẫn có thể chấp nhận vì mục tiêu chính là xác định đúng lỗi và vùng linh kiện thiếu để hiển thị, không phải đo kích thước linh kiện chính xác.

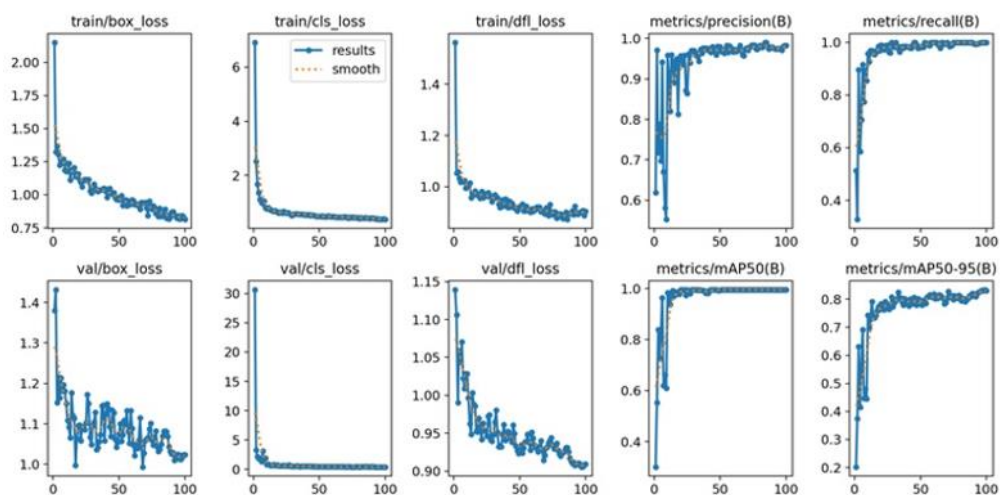
4.3.2. YOLO Detection sau khi bổ sung ảnh WGAN-GP

Chỉ số	Dữ liệu gốc	Có WGAN-GP	Chênh lệch	Đánh giá
Precision	98,82%	98,26%	Giảm 0,56 điểm %	Vẫn rất cao, nhưng ảnh bổ sung không làm precision tăng.
Recall	100%	100%	Không đổi	Tốt. Mô hình vẫn không bỏ sót lỗi trong tập đánh giá.
mAP50	99,50%	99,50%	Không đổi	Đã gần bão hòa ở ngưỡng IoU 0,5.
mAP50-95	82,56%	83,13%	+0,57 điểm %	Cải thiện nhẹ ở độ chính xác bounding box.
Val box loss cuối	0,97425	1,02355	Tăng nhẹ	Bounding box validation không tốt hơn rõ rệt.
Val cls loss cuối	0,40740	0,41673	Tăng nhẹ	Khả năng phân lớp gần như tương đương.
Val dfl loss cuối	0,90971	0,90934	Gần như không đổi	Độ ổn định phân bố box gần tương đương.

Detection - dữ liệu gốc



Detection - dữ liệu có GAN



Hình 4.4. Sơ đồ kết quả huấn luyện YOLO Detection trước và sau khi bổ sung WGAN-GP

Dự đoán Detection - dữ liệu gốc



Dự đoán Detection - có GAN



Hình 4.5. Kết quả dự đoán của YOLO Detection trên ảnh validation

Sau khi bổ sung ảnh WGAN-GP, thông số mAP50-95 tăng nhẹ từ 82,56% lên 83,13%, trong khi Recall và mAP50 giữ nguyên. Điều này cho thấy dữ liệu bổ sung không làm mô hình kém đi và có tác động nhỏ đến độ khớp bounding box. Tuy nhiên, Precision giảm nhẹ và val box loss tăng nhẹ, nên không thể kết luận WGAN-GP cải thiện Detection một cách rõ ràng.

➔ Ảnh WGAN-GP sau lọc có thể dùng như dữ liệu hỗ trợ, nhưng hiệu quả tăng thêm còn hạn chế.

Dù mức cải thiện do WGAN-GP không lớn, YOLO Detection vẫn là hướng phù hợp nhất. Lý do chính nằm ở dạng đầu ra: mô hình trả về class, confidence và bounding box. Các thông tin này được dùng trực tiếp trong Unity để vẽ khung lỗi, hiển thị tên lỗi và đưa PCB vào khay tương ứng. Đây là điểm mà Classification không đáp ứng được.

4.4. SO SÁNH VÀ LỰA CHỌN MÔ HÌNH TRIỂN KHAI

Mô hình	Kết quả chính	Điểm đạt được	Hạn chế
YOLO Classification	Top-1 Accuracy 92,71%; Accuracy đánh giá 96,35%	Tốt với lỗi đơn, dễ huấn luyện.	Không xác định vị trí lỗi; khó mở rộng khi một PCB có nhiều lỗi.
YOLO Classification + WGAN-GP	Top-1 Accuracy 94,93%; Accuracy đánh giá 97,46%	Có cải thiện nhẹ về accuracy và validation loss.	Cải thiện chưa giải quyết được nhược điểm đầu ra một nhãn/ảnh.
YOLO Detection	Precision 98,82%; Recall 100%; mAP50 99,50%; mAP50-95 82,56%	Phát hiện tốt và có bounding box để hiển thị lỗi.	Bounding box ở ngưỡng cao còn có thể cải thiện.

YOLO Detection + WGAN-GP	Recall 100%; mAP50 99,50%; mAP50-95 83,13%	Giữ được chất lượng phát hiện, mAP50-95 tăng nhẹ.	WGAN-GP chỉ nên xem là hỗ trợ dữ liệu, không phải nguyên nhân chính giúp hệ thống hoạt động.
--------------------------	--	---	--

Từ các kết quả trên, mô hình được chọn cho hệ thống là YOLO Detection. Nếu xét riêng độ chính xác, Classification cũng đạt kết quả cao. Tuy nhiên, hệ thống kiểm tra PCB cần biết vị trí linh kiện bị thiếu để hiển thị và phục vụ quy trình phân loại/sửa lỗi. Detection đáp ứng đúng yêu cầu này. Với trường hợp PCB có nhiều lỗi, Detection cũng tự nhiên hơn vì mỗi lỗi là một object riêng, không cần tạo thêm các lớp tổ hợp như Classification.

Phiên bản Detection có bổ sung WGAN-GP được sử dụng như hướng triển khai chính trong đề án vì vẫn giữ Recall 100%, mAP50 99,50% và có mAP50-95 nhỉnh hơn dữ liệu gốc. Tuy vậy WGAN-GP chỉ đóng vai trò bổ sung dữ liệu sau lọc nhằm mở rộng tập dữ liệu gốc bị hạn chế.

4.5. KẾT QUẢ CHẠY MÔ PHỎNG TRÊN UNITY 3D

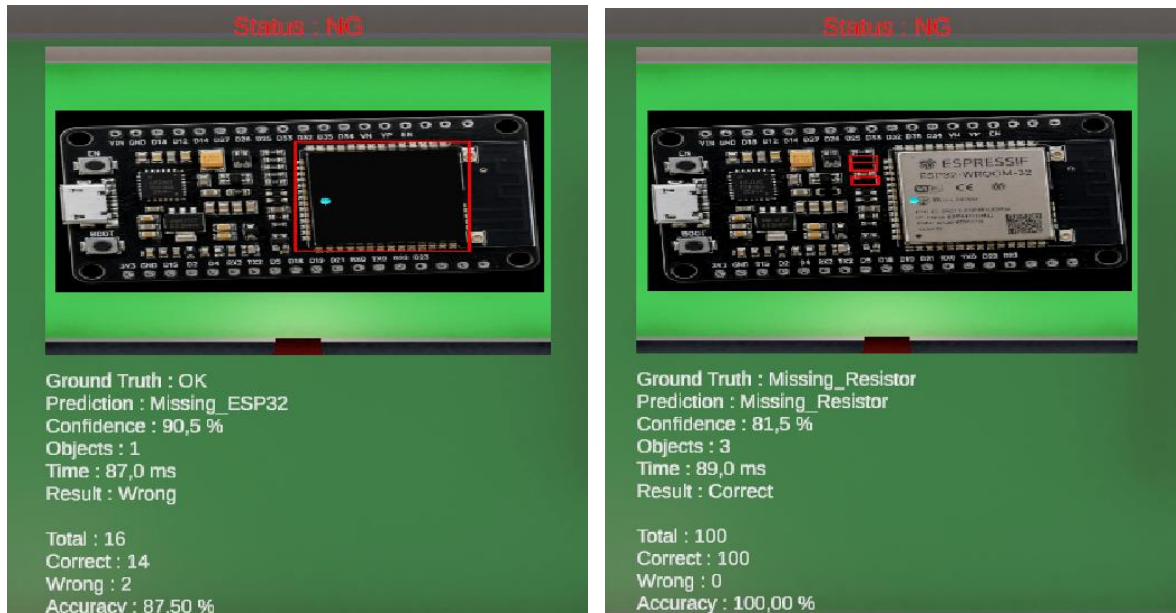
Sau khi huấn luyện, mô hình YOLO Detection được đưa vào Unity để kiểm tra luồng vận hành. Do hệ thống không mô phỏng vật lý chi tiết của băng tải và xi lanh, phần đánh giá tập trung vào logic hệ thống: PCB có được chụp ảnh đúng thời điểm hay không, mô hình có trả kết quả hay không, giao diện có hiển thị đúng OK/NG hay không và kết quả có được truyền cho khối phân loại hay không.

Nội dung kiểm tra	Kết quả	Nhận xét
Hệ thống sinh ảnh ngẫu nhiên	Đạt	PCB đã lấy được mẫu ngẫu nhiên từ 1 trong 6 file class được chia trong Asset
Chụp ảnh PCB tại vùng kiểm tra	Đạt	Camera Unity đã thu ảnh PCB và gửi cho mô hình Detection.
Hiển thị kết quả nhận dạng	Đạt	Giao diện hiển thị trạng thái OK/NG, tên lỗi, confidence và thời gian xử lý.
PCB lỗi	Đạt	Mô hình phát hiện số object lỗi, vẽ bounding box và hiển thị trạng thái NG.
PCB đạt yêu cầu	Đạt	Không có object lỗi vượt ngưỡng hoặc không phát hiện object lỗi, giao diện hiển thị trạng thái OK.
Kiểm tra độ chính xác nhận dạng Camera AI	Đạt	Bằng cách lưu thông tin tên class PCB được lấy từ file class rồi so sánh với kết quả nhận dạng → Hiển thị kết quả result là Correct/Wrong và bộ đếm % dựa trên số lượng Correct/Wrong và tổng số PCB kiểm tra → Tỷ lệ chính xác 100% với tập ảnh đã kiểm tra

Liên kết với logic phân loại	Đạt	Kết quả nhận dạng được dùng làm đầu vào cho script phân loại PCB vào khay.
Phân loại vào Sorting	Đạt	PCB được chia vào đúng các Tray



Hình 4.6. Giao diện màn hình UI Unity khi phân loại PCB



Hình 4.7. Giao diện màn hình UI Unity thống kê số lượng PCB train

Kết quả chạy mô phỏng cho thấy luồng xử lý cơ bản đã hoạt động đúng: PCB được lấy ngẫu nhiên, di chuyển đến vùng kiểm tra, ảnh được đưa vào mô hình, kết quả được hiển thị và trạng thái PCB được dùng cho bước phân loại. Với PCB lỗi, giao diện hiển thị số object phát hiện và bounding box. Với PCB_OK, hệ thống không phát hiện lỗi và chuyển

trạng thái OK. Đây là kết quả phù hợp với mục tiêu của đề án ở mức mô phỏng logic. Đồng thời cũng thống kê tỉ lệ phân định của mô hình khi tích hợp vào Unity.

- Kiểm tra với tập ảnh test của mô hình :
+ 100 lần test đều đạt kết quả đúng với phân loại : tỉ lệ Accuracy 100%

→ Tuy nhiên, kết quả Unity hiện tại chưa thể đánh giá cho một dây chuyền thật. Các yếu tố như sai số cơ khí, rung động, thay đổi ánh sáng, trễ truyền thông và độ ổn định camera chưa được kiểm tra. Vì vậy, có thể kết luận hệ thống đã chứng minh được nguyên lý khi tích hợp AI với mô phỏng Unity, nhưng chưa đủ để khẳng định khả năng làm việc trong môi trường công nghiệp thực tế.

4.6. KẾT LUẬN CHƯƠNG

Các kết quả thực nghiệm cho thấy WGAN-GP có khả năng sinh ảnh mang dạng tổng thể của PCB, nhưng chất lượng chi tiết chưa đủ ổn định để dùng như dữ liệu thật nếu không lọc, đồng thời do hạn chế dataset đầu vào nên mô hình WGAN-GP chưa thể sinh ảnh như kỳ vọng. Với YOLO Classification, mô hình đạt accuracy cao trên tập lỗi đơn, nhưng hạn chế lớn là chỉ có một nhãn cho toàn ảnh và không định vị được vùng lỗi. Với YOLO Detection, các chỉ số Precision, Recall và mAP50 đều cao; mô hình phát hiện được lỗi, trả về bounding box và phù hợp hơn với giao diện Unity.

Dữ liệu WGAN-GP giúp một số chỉ số tăng nhẹ, đặc biệt là Top-1 Accuracy của Classification và mAP50-95 của Detection. Tuy nhiên mức cải thiện không lớn → Vì vậy, trong đề án này không nên coi WGAN-GP là yếu tố quyết định. Mô hình chính được lựa chọn là YOLO Detection vì đầu ra của nó phù hợp với bài toán kiểm tra PCB tự động: biết lỗi gì, lỗi nằm ở đâu và có thể dùng kết quả đó cho hiển thị cũng như phân loại sản phẩm trong Unity.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. KẾT LUẬN CHUNG

5.1.1. Tóm tắt thành tựu của đề tài

Đồ án đã xây dựng được một hệ thống mô phỏng kiểm tra và phân loại PCB thiếu linh kiện trên bo mạch ESP32 DevKit. Hệ thống được triển khai theo hướng kết hợp giữa xử lý ảnh, mô hình học sâu và mô phỏng logic vận hành trong Unity 3D. Trong đó, mô hình AI thực hiện nhiệm vụ nhận dạng lỗi từ ảnh PCB, còn Unity được dùng để mô phỏng quá trình đưa sản phẩm qua vùng kiểm tra và phân loại vào khay tương ứng.

Về dữ liệu, đề tài đã xây dựng bộ ảnh PCB gồm các trạng thái: PCB_OK, Missing_USB, Missing_ESP32, Missing_Resistor, Missing_Capacitor và Missing_CP2102. Bộ dữ liệu này được chuẩn bị theo hai dạng. Dạng thứ nhất dùng cho YOLO Classification, trong đó mỗi ảnh được gán một nhãn trạng thái. Dạng thứ hai dùng cho YOLO Detection, trong đó các vùng linh kiện bị thiếu được gán nhãn bằng bounding box. Cách chuẩn bị này giúp đề tài có cơ sở để so sánh hai hướng nhận dạng khác nhau.

Đề tài cũng đã thử nghiệm WGAN-GP để sinh thêm ảnh PCB. Kết quả cho thấy WGAN-GP có thể tạo ra một số ảnh mang đặc điểm tổng thể của bo mạch, nhưng chất lượng ảnh chưa ổn định. Một số ảnh bị mờ, sai chi tiết hoặc không giữ tốt cấu trúc linh kiện. Vì vậy, ảnh sinh không được sử dụng trực tiếp toàn bộ, mà chỉ có thể dùng sau khi lọc chất lượng. Kết quả này cho thấy WGAN-GP có thể hỗ trợ mở rộng dữ liệu, nhưng chưa đủ tin cậy để thay thế ảnh thật trong bài toán kiểm tra PCB.

Về phần nhận dạng, đề tài đã huấn luyện và đánh giá hai hướng YOLO Classification và YOLO Detection. YOLO Classification cho kết quả tốt trong trường hợp mỗi ảnh PCB chỉ có một trạng thái lỗi rõ ràng. Tuy nhiên, mô hình này chỉ trả về một nhãn cho toàn ảnh nên không xác định được vị trí lỗi. Khi xét đến trường hợp một PCB có thể thiếu nhiều linh kiện cùng lúc, hướng Classification sẽ gặp khó khăn vì phải tạo thêm nhiều lớp tổ hợp lỗi.

YOLO Detection phù hợp hơn với yêu cầu của đề tài. Kết quả đầu ra của mô hình gồm tên lỗi, độ tin cậy và tọa độ bounding box. Nhờ đó, hệ thống không chỉ biết PCB thuộc trạng thái nào, mà còn biết vùng lỗi nằm ở đâu trên ảnh. Thông tin này phù hợp với bài toán kiểm tra tự động, đặc biệt khi cần hiển thị lỗi trên giao diện hoặc định hướng xử lý lỗi sau kiểm tra.

Trong Unity 3D, đồ án đã mô phỏng được luồng vận hành cơ bản: PCB được khởi tạo, di chuyển đến vùng kiểm tra, camera chụp ảnh, mô hình YOLO Detection nhận dạng lỗi, sau đó kết quả được dùng để phân loại sản phẩm. Các chuyển động trong mô phỏng được điều khiển bằng script C#, không mô phỏng chi tiết lực, ma sát hoặc va chạm vật lý. Cách làm này phù hợp với phạm vi đồ án vì mục tiêu chính là kiểm chứng logic tích hợp giữa mô hình AI và hệ thống phân loại.

5.1.2. Đánh giá mức độ hoàn thành mục tiêu

So với mục tiêu ban đầu, đề án đã hoàn thành các nội dung chính.

Đầu tiên, đề tài đã xây dựng được bộ dữ liệu PCB phục vụ cho cả bài toán Classification và Detection. Các lớp lỗi được lựa chọn đều là lỗi thiếu linh kiện, phù hợp với phạm vi kiểm tra bằng ảnh và dễ quan sát từ góc nhìn camera phía trên.

Tiếp theo, đề tài đã huấn luyện được các mô hình YOLO trên dữ liệu gốc và dữ liệu có bổ sung ảnh sinh bởi WGAN-GP. Kết quả thực nghiệm cho thấy việc bổ sung ảnh sinh chỉ có ý nghĩa khi ảnh được lọc kỹ. Nếu ảnh sinh bị sai cấu trúc, mô hình có thể học nhầm đặc trưng và làm giảm độ tin cậy khi kiểm thử.

Đề án cũng đã chỉ ra được sự khác nhau giữa Classification và Detection trong bài toán kiểm tra PCB. Classification có ưu điểm là dữ liệu dễ chuẩn bị hơn, nhưng đầu ra chưa đủ cho hệ thống kiểm tra tự động. Detection yêu cầu gán nhãn bounding box tốn công hơn, nhưng đổi lại mô hình cung cấp được vị trí lỗi. Đây là lý do YOLO Detection được chọn làm mô hình chính để tích hợp vào Unity.

Phần mô phỏng Unity đã thể hiện được quy trình vận hành ở mức logic. Hệ thống có thể nhận ảnh PCB, gọi mô hình YOLO Detection, hiển thị kết quả và đưa sản phẩm vào khay tương ứng. Như vậy, kết quả của đề án không dừng lại ở việc huấn luyện mô hình, mà đã đưa mô hình vào một quy trình kiểm tra cụ thể.

Tuy nhiên, đề án vẫn còn một số nội dung chưa hoàn thiện. Dữ liệu thực tế chưa nhiều, đặc biệt là dữ liệu lỗi thật trong môi trường sản xuất. Unity mới mô phỏng luồng xử lý bằng script, chưa đánh giá được các yếu tố cơ khí như sai số vị trí, độ trễ cơ cấu phân loại hoặc tốc độ băng tải thực. Ngoài ra, WGAN-GP mới dừng ở mức khảo sát hỗ trợ dữ liệu, chưa tạo được ảnh PCB đủ ổn định để dùng tự động.

5.1.3. Giá trị khoa học và thực tiễn của đề tài

Giá trị chính của đề tài nằm ở việc xây dựng và đánh giá một quy trình kiểm tra PCB tự động ở mức mô phỏng. Đề tài không chỉ huấn luyện một mô hình nhận dạng ảnh, mà còn đặt mô hình đó vào một hệ thống có luồng xử lý rõ ràng: thu ảnh, nhận dạng lỗi, hiển thị kết quả và phân loại sản phẩm.

Về mặt kỹ thuật, kết quả so sánh giữa YOLO Classification và YOLO Detection cho thấy Detection phù hợp hơn với bài toán thiếu linh kiện. Đối với hệ thống kiểm tra PCB, việc biết vị trí linh kiện bị thiếu quan trọng hơn việc chỉ biết nhãn tổng thể của ảnh. Nhận xét này là cơ sở để lựa chọn mô hình trong các hệ thống kiểm tra sản phẩm có yêu cầu định vị lỗi.

Phần WGAN-GP giúp đánh giá thêm khả năng mở rộng dữ liệu khi số lượng ảnh lỗi còn hạn chế. Kết quả thực nghiệm cho thấy ảnh sinh có thể dùng làm nguồn tham khảo, nhưng không nên đưa thẳng vào huấn luyện nếu chưa kiểm tra chất lượng. Đây là điểm cần lưu ý khi áp dụng mô hình sinh ảnh cho các bài toán công nghiệp, nơi hình dạng và vị trí chi tiết phải chính xác.

Về mặt thực tiễn, hệ thống mô phỏng có thể dùng để minh họa nguyên lý kiểm tra PCB tự động trong môi trường học tập và nghiên cứu. Nếu được bổ sung dữ liệu thật, camera thật và cơ cấu phân loại vật lý, hướng này có thể phát triển tiếp thành một hệ thống kiểm tra PCB quy mô nhỏ.

5.2. PHÂN TÍCH ƯU ĐIỂM VÀ HẠN CHẾ

5.2.1. Ưu điểm nổi bật của hệ thống

Hệ thống có cấu trúc tương đối rõ ràng. Các khối chức năng gồm tạo PCB, di chuyển đến vùng kiểm tra, chụp ảnh, nhận dạng bằng YOLO Detection và phân loại sản phẩm được tách riêng. Cách tổ chức này giúp việc kiểm tra, sửa lỗi và mở rộng hệ thống thuận lợi hơn.

Ưu điểm quan trọng nhất là việc chọn YOLO Detection làm hướng chính. Mô hình Detection trả về cả loại lỗi và vị trí lỗi, phù hợp với yêu cầu kiểm tra thiếu linh kiện. Khi tích hợp vào Unity, bounding box có thể được dùng để hiển thị vùng lỗi trực tiếp trên ảnh PCB. Đây là điểm mà YOLO Classification không đáp ứng được.

Hệ thống cũng có khả năng mở rộng tốt hơn khi xét đến trường hợp PCB có nhiều lỗi. Với Detection, mỗi linh kiện thiếu được xem như một đối tượng riêng. Nếu một PCB thiếu nhiều linh kiện, mô hình có thể trả về nhiều bounding box trong cùng một ảnh. Trong khi đó, Classification sẽ phải tạo thêm các lớp tổ hợp lỗi, làm dữ liệu phức tạp và khó quản lý.

Một ưu điểm khác là đề tài có thử nghiệm WGAN-GP để đánh giá khả năng bổ sung dữ liệu. Dù kết quả ảnh sinh chưa hoàn toàn ổn định, phần này cho thấy rõ giới hạn của dữ liệu nhân tạo trong bài toán PCB. Đây là nhận xét thực tế, vì ảnh PCB yêu cầu giữ đúng bố cục linh kiện, không giống các bài toán sinh ảnh thông thường.

Mô phỏng Unity cũng giúp kết quả trực quan hơn. Người xem có thể theo dõi quá trình PCB đi qua vùng camera, nhận dạng lỗi và được đưa vào khay. Điều này làm rõ hơn vai trò của mô hình AI trong một quy trình kiểm tra sản phẩm, thay vì chỉ đánh giá mô hình bằng các chỉ số huấn luyện.

5.2.2. Hạn chế còn tồn tại

Hạn chế đầu tiên là dữ liệu còn ít và chưa đủ đa dạng. Phần lớn ảnh được xây dựng trong điều kiện tương đối cố định về góc chụp, ánh sáng và vị trí PCB. Điều này giúp mô hình dễ học hơn, nhưng chưa phản ánh đầy đủ điều kiện làm việc thực tế. Khi đưa sang camera thật, mô hình có thể bị ảnh hưởng bởi độ sáng, phản xạ bề mặt, sai lệch vị trí hoặc nhiễu ảnh.

Hạn chế thứ hai là phạm vi lỗi còn hẹp. Đề tài mới tập trung vào lỗi thiếu linh kiện. Các lỗi khác như cầu hàn, thiếu hàn, lệch linh kiện, sai chiều linh kiện hoặc đứt mạch chưa được xử lý. Đây là các lỗi thường gặp trong sản xuất PCB nhưng yêu cầu dữ liệu và phương pháp đánh giá phức tạp hơn.

Hạn chế thứ ba nằm ở WGAN-GP. Mặc dù mô hình có thể sinh thêm ảnh nhưng chất lượng ảnh chưa đồng đều, một số ảnh không giữ được chi tiết linh kiện hoặc bị biến dạng

vùng mạch. Vì vậy, ảnh sinh chỉ có thể dùng sau bước lọc thủ công. Điều này làm giảm tính tự động của quá trình mở rộng dữ liệu.

Hạn chế thứ tư là Unity mới được dùng để mô phỏng logic vận hành, chưa phải mô phỏng vật lý hoàn chỉnh. Băng tải, cảm biến và Cylinder chủ yếu đóng vai trò biểu diễn quy trình. Hệ thống chưa xét đến vận tốc băng tải thực, sai số dừng PCB, độ trễ chụp ảnh, thời gian phản hồi của cơ cấu phân loại hoặc khả năng va chạm giữa sản phẩm và cơ cấu gạt.

Hạn chế cuối cùng là hệ thống chưa được thử nghiệm trên phần cứng thật. Vì vậy, các kết quả hiện tại mới chứng minh được nguyên lý và khả năng tích hợp mô hình vào mô phỏng. Để đánh giá sát thực tế hơn, cần kiểm tra thêm với camera thật, ánh sáng thật và cơ cấu phân loại vật lý.

5.3. HƯỚNG PHÁT TRIỂN TRONG TƯƠNG LAI

- Mở rộng bộ dữ liệu. Cần bổ sung thêm ảnh PCB thật trong nhiều điều kiện khác nhau, chẳng hạn thay đổi ánh sáng, vị trí đặt PCB, khoảng cách camera và nền ảnh. Dữ liệu càng gần với điều kiện vận hành thực tế thì mô hình càng có khả năng nhận dạng ổn định hơn.
- Mở rộng hệ thống sang các dạng lỗi khác ngoài thiếu linh kiện. Các lỗi như cầu hàn, thiếu hàn, lệch linh kiện hoặc sai chiều linh kiện có giá trị thực tế cao hơn, nhưng cũng khó nhận dạng hơn. Với các lỗi nhỏ, có thể cần kết hợp YOLO Detection với ảnh độ phân giải cao hơn hoặc mô hình segmentation để tách vùng lỗi rõ hơn.
- Phân dữ liệu sinh, có thể thử các mô hình có điều kiện thay cho WGAN-GP thông thường. Nếu kiểm soát được đầu vào, mô hình sinh ảnh có thể tạo lỗi ở đúng vị trí mong muốn và giữ cấu trúc PCB ổn định hơn.
- Phát triển Unity theo hướng mô phỏng sát hệ thống thật hơn. Băng tải có thể được gán vận tốc cụ thể, cảm biến có vùng phát hiện rõ ràng, Cylinder có hành trình và thời gian tác động. Khi đó, hệ thống có thể đánh giá thêm thời gian chu kỳ, độ trễ xử lý và khả năng đồng bộ giữa nhận dạng và phân loại.
- Xây dựng quy trình kiểm tra - sửa lỗi - kiểm tra lại. Khi YOLO Detection phát hiện một linh kiện bị thiếu, PCB có thể được đưa đến công đoạn sửa tương ứng. Sau khi sửa, PCB quay lại vùng kiểm tra. Nếu vẫn còn lỗi, quy trình tiếp tục lặp lại; nếu không còn lỗi, PCB được phân loại là đạt yêu cầu. Hướng này tận dụng được ưu điểm của Detection, vì mô hình có thể phát hiện nhiều lỗi riêng biệt trên cùng một ảnh.
- Xây dựng hệ thống trên sản phẩm thực tế với một mô hình thử nghiệm có thể gồm camera, băng tải mini, cảm biến quang, máy tính xử lý và cơ cấu gạt sản phẩm. Khi có dữ liệu vận hành thực tế, các kết quả trong Unity sẽ được kiểm chứng rõ hơn.

5.4. LỜI KẾT

Đồ án đã hoàn thành việc xây dựng và đánh giá một hệ thống mô phỏng kiểm tra PCB thiếu linh kiện bằng Computer Vision và YOLO. Kết quả thực nghiệm cho thấy YOLO Detection phù hợp hơn YOLO Classification đối với bài toán này, do mô hình có thể xác định cả loại lỗi và vị trí lỗi trên ảnh.

WGAN-GP được khảo sát như một phương pháp hỗ trợ mở rộng dữ liệu. Kết quả cho thấy hướng này có thể tạo thêm ảnh tham khảo, nhưng chất lượng ảnh sinh chưa đủ ổn định để sử dụng tự động. Vì vậy, trong phạm vi đồ án, WGAN-GP được xem là phần hỗ trợ, còn mô hình YOLO Detection vẫn là thành phần chính của hệ thống.

Mô phỏng Unity đã thể hiện được luồng kiểm tra và phân loại PCB ở mức logic. Hệ thống chưa mô phỏng đầy đủ cơ khí và chưa chạy trên phần cứng thật, nhưng đã cho thấy cách kết hợp giữa mô hình nhận dạng ảnh và quy trình phân loại sản phẩm.

Qua quá trình thực hiện, đề tài giúp củng cố kiến thức về xử lý ảnh, học sâu, xây dựng dữ liệu, lập trình Python, lập trình C# và tổ chức một hệ thống mô phỏng Cơ điện tử. Những kết quả đạt được là cơ sở để tiếp tục phát triển hệ thống theo hướng dữ liệu thực tế hơn, mô hình nhận dạng ổn định hơn và tiến tới thử nghiệm trên thiết bị thật.

TÀI LIỆU THAM KHẢO

- [1] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y. (2014). Generative Adversarial Nets. Advances in Neural Information Processing Systems. <https://arxiv.org/abs/1406.2661>
- [2] Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., Courville, A. (2017). Improved Training of Wasserstein GANs. Advances in Neural Information Processing Systems. <https://arxiv.org/abs/1704.00028>
- [3] Redmon, J., Divvala, S., Girshick, R., Farhadi, A. (2015). You Only Look Once: Unified, Real-Time Object Detection. <https://arxiv.org/abs/1506.02640>
- [4] Goodfellow, I., Bengio, Y., Courville, A. (2016). Deep Learning. MIT Press. <https://www.deeplearningbook.org/>
- [5] Ultralytics. Object Detection - Ultralytics YOLO Documentation. <https://docs.ultralytics.com/tasks/detect/>
- [6] Ultralytics. Image Classification - Ultralytics YOLO Documentation. <https://docs.ultralytics.com/tasks/classify/>
- [7] Ultralytics. Predict and Results API Documentation. <https://docs.ultralytics.com/modes/predict/>
- [8] Roboflow. Roboflow Documentation. <https://docs.roboflow.com/>
- [9] Ciaglia, F., Zuppichini, F. S., Guerrie, P., McQuade, M., Solawetz, J. (2022). Roboflow 100: A Rich, Multi-Domain Object Detection Benchmark. <https://arxiv.org/abs/2211.13523>
- [10] Unity Technologies. Unity User Manual. <https://docs.unity3d.com/Manual/index.html>
- [11] Szeliski, R. (2022). Computer Vision: Algorithms and Applications, 2nd Edition. Springer. <https://szeliski.org/Book/>
- [12] Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.
- [13] Prince, S. J. D. (2012). Computer Vision: Models, Learning, and Inference. Cambridge University Press.
- [14] Duda, R. O., Hart, P. E., Stork, D. G. (2001). Pattern Classification, 2nd Edition. Wiley.

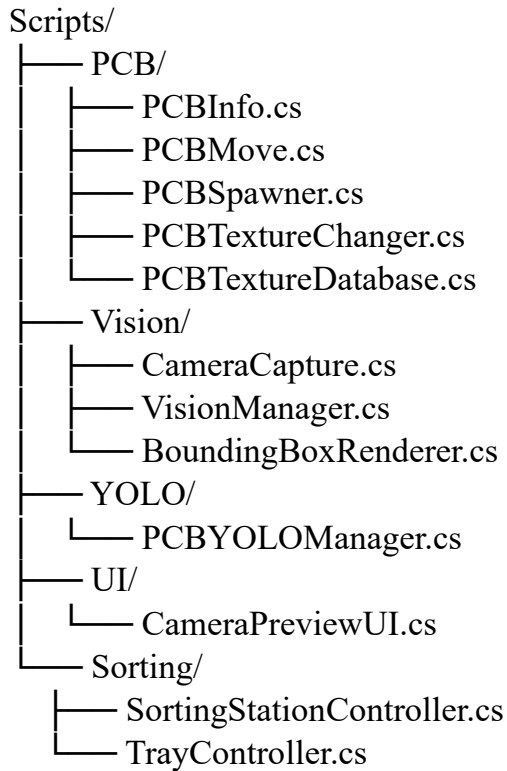
PHỤ LỤC

Mã nguồn đầy đủ của project được lưu tại GitHub:

https://github.com/MinhHoang43/DATN_Computer-AI-Simulation

Link video mô phỏng : <https://youtu.be/3bqB-Kqhzoo>

- Cấu trúc thư mục script Unity và chức năng tương ứng



Nhóm	Script	Chức năng
PCB	PCBInfo.cs	Lưu thông tin và trạng thái của từng PCB trong mô phỏng. Script quản lý nhãn thật, kết quả dự đoán, độ tin cậy, trạng thái OK/NG và kết quả đúng/sai sau khi so sánh dự đoán với nhãn thật. Script cũng chuẩn hóa tên lớp để hệ thống so sánh ổn định giữa các cách đặt tên khác nhau.
PCB	PCBMove.cs	Điều khiển PCB di chuyển trên băng tải. Script theo dõi vị trí PCB so với điểm kiểm tra và các trạm phân loại, gửi yêu cầu kiểm tra sang VisionManager, dừng hoặc cho PCB chạy tiếp, đồng thời hỗ trợ di chuyển PCB đến vị trí đích trong quá trình phân loại.
PCB	PCBSpawner.cs	Tạo PCB mới trong scene theo khoảng thời gian cài đặt. Script lấy ngẫu nhiên texture PCB từ PCBTextureDatabase, gán texture lên PCB và gán

		nhân thật vào PCBInfo để phục vụ đánh giá dự đoán.
PCB	PCBTextureChanger.cs	Thay đổi ảnh hiển thị trên bề mặt PCB. Script tìm object con PCB_Texture, lấy MeshRenderer và gán texture mới vào material.
PCB	PCBTextureDatabase.cs	Load và quản lý ảnh PCB trong Assets/Resources/PCBImages. Script đọc ảnh theo từng thư mục lớp, lưu ảnh xạ texture - nhân thật, hỗ trợ nhiều kiểu đặt tên folder và cung cấp ảnh ngẫu nhiên cho PCBSpawner.
Vision	CameraCapture.cs	Chụp ảnh từ camera trong Unity bằng RenderTexture, chuyển ảnh thành Texture2D kích thước 640x640 và trả ảnh cho mô hình YOLO. Script có thể lưu ảnh ra máy nếu bật chế độ saveDataset.
Vision	VisionManager.cs	Điều phối toàn bộ quá trình kiểm tra PCB. Script dừng PCB tại vùng kiểm tra, chụp ảnh, gọi PCBYOLOManager để nhận dạng, đo thời gian suy luận, cập nhật PCBInfo, hiển thị kết quả lên UI và cho PCB chạy tiếp.
Vision	BoundingBoxRenderer.cs	Vẽ bounding box lên ảnh kết quả. Script nhận ảnh gốc và danh sách detection, chuyển tọa độ box sang hệ pixel của ảnh rồi vẽ khung đỏ quanh vùng lỗi.
YOLO	PCBYOLOManager.cs	Load mô hình YOLO dạng ONNX, load labels, chuyển ảnh đầu vào thành tensor, chạy inference bằng Unity.InferenceEngine trên GPUCompute, áp dụng NMS và trả về danh sách detection gồm class, confidence và bounding box.
UI	CameraPreviewUI.cs	Hiển thị ảnh camera hoặc ảnh đã vẽ bounding box trên giao diện. Script cập nhật trạng thái OK/NG, ground truth, prediction, confidence, số object, thời gian xử lý, kết quả đúng/sai và thống kê accuracy tạm thời.
Sorting	SortingStationController.cs	Điều khiển trạm phân loại. Script so sánh lớp lỗi của PCB với targetClass của trạm, dừng PCB, mô phỏng Cylinder đẩy PCB sang nhánh phân loại, đưa PCB đến entry point và chuyển PCB vào khay.
Sorting	TrayController.cs	Quản lý PCB trong khay. Script tự load các slot, đưa PCB vào vị trí trống, giới hạn số lượng PCB theo capacity và có thể xóa PCB cũ nhất khi khay đầy.